

Bachelor's Thesis

A Secure Self-Hosted Platform for Programming Assignments

GitLab-Based Provisioning, CI Assessment, and Infrastructure Automation

Software version: [HandIn](#) 0.9.0-beta

Author

Andreas Auer

`andreas.auer@stud.plus.ac.at`

Supervisor

Assoc. Prof. Dipl.-Ing. Dr. Andreas Naderlinger

`andreas.naderlinger@plus.ac.at`

University of Salzburg

Department of Computer Science

Submitted on 12 August 2026

Abstract

A programming course hands the same exercise to every student: starter code to build on, automated tests the submission has to satisfy, and a deadline after which nothing further counts. Administering that by hand means creating a repository for each participant, granting and then withdrawing access at exactly the right moment, running the tests on every submission, and keeping the grading logic out of reach throughout. The effort grows with enrolment, and one misplaced permission can expose the tests or quietly extend a deadline for a single student. This thesis presents HandIn, a self-hosted platform that automates this workflow around a self-managed GitLab instance. Courses, assignments, repositories, staff, and deadlines are represented directly through GitLab rather than duplicated in a separate database. A web application gives each role a dedicated interface, while students can work either in the browser or with a local Git client. The central design problem is not simply how to automate GitLab, but how to do so without giving the public-facing application unrestricted control over it. HandIn addresses this by separating the decision of what may happen from the privileged operations needed to carry it out. The result is a workflow in which teachers prepare an assignment once, students receive their own repositories, submissions are tested automatically, and access ends with the deadline. The deployed system supports this lifecycle for Java/Maven and Python and is validated through unit, integration, security, and operational testing.

Acknowledgements

I would like to thank my supervisor, Assoc. Prof. Dipl.-Ing. Dr. Andreas Naderlinger, for his patient guidance and thoughtful feedback throughout this project. His input shaped both the technical direction of the work and the way it is presented in these pages, and the freedom he gave me to pursue an ambitious scope is what allowed the platform to grow into what it is today.

My thanks also go to the department's IT staff for providing and hosting the production servers on which [HandIn](#) has been tested and now runs. Without that infrastructure – and the operational patience it took to accommodate an early-stage system – the platform would have remained a prototype rather than a service exercised against real coursework.

Contents

Abstract	i
Acknowledgements	ii
Glossary of Terms	vi
List of Acronyms	x
1 Introduction	1
1.0.1 Motivation	1
1.0.2 Problem Statement	2
1.0.3 Thesis Scope	3
1.0.4 Research Questions	3
1.0.5 Contributions	3
1.0.6 Structure of the Thesis	4
2 Background	5
2.1 GitLab as Code Host and Automation Platform	5
2.2 Continuous Integration for Programming Assignments	7
2.3 OAuth 2.0 and Server-Side Sessions	7
2.4 Infrastructure as Code with Ansible	8
2.5 Containerisation and Runner Isolation	9
2.6 Redis for Transient Coordination	9
3 Requirements and Design Principles	11
3.0.1 Stakeholders and Roles	11
3.0.2 Use Cases	12
3.0.3 Functional Requirements	13
3.0.4 Non-Functional Requirements	13
3.0.5 Derived Design Principles	14
4 System Architecture	16
4.1 Overview	16

4.2	Physical Deployment	16
4.3	Per-Host Internal Structure	17
4.4	Logical Components	19
4.5	Source-of-Truth Model	20
4.6	User-Facing Workflows	20
4.7	Implementation Notes by Component	21
5	Focus 1: Security and Trust Boundaries	23
5.1	Threat Model	23
5.2	Credential Separation	24
5.3	Authentication, Sessions, and CSRF	24
5.4	Server-Side Role and Course Authorisation	25
5.5	Request Control	25
5.6	Invitation and Rejoin Token Security	26
5.7	Admin Daemon Isolation	27
5.8	Runner Trust Boundary	27
5.9	Test-Mode Security	27
5.10	Security Evaluation	28
6	Focus 2: GitLab-Based Assignment Lifecycle	30
6.1	Namespace Design	30
6.2	Repository Files as Policy	31
6.3	Generated Runtime State	31
6.4	Provisioner Pipeline Architecture	32
6.5	Course and Assignment Creation	32
6.6	Invitation Claim Lifecycle	33
6.7	Deadline Lifecycle	35
6.8	Teacher, Tutor, and Proctor Workflows	35
7	Deployment and Operations with Ansible	37
7.1	Inventory Model	37
7.2	Playbook Ordering and Bootstrap State	37
7.3	GitLab Play	38
7.4	CI Play	39
7.5	Web Play	40
7.6	Firewall, TLS, and Network Boundaries	41
7.7	Deployment Pipeline	42
8	CI-Based Assessment and Reproducibility	43
8.1	Language Templates	43
8.2	Student Compile and Test Jobs	43
8.3	Private Grader and Package Registry Flow	44

8.4	Per-Assignment Docker Images	45
8.5	Caching and Performance Considerations	46
9	Evaluation	48
9.1	Evaluation Strategy	48
9.2	Automated Test Coverage	48
9.3	Integration Test Playbook	49
9.4	Security Evaluation Matrix	50
9.5	Operational Evaluation	52
9.6	Capacity and Resource Usage	52
9.6.1	Persistent Storage for a Standard Course	53
9.6.2	Idle Baseline Over Time	53
9.6.3	Resource Use by Run Scenario	54
9.7	Limitations of the Evaluation	54
10	Discussion	55
10.1	Design Trade-Offs	55
10.2	Known Limitations	55
10.3	Future Work	56
11	Conclusion	59
A	Additional Diagrams and Reference Tables	60
A.1	Namespace Structure Trees	60
A.2	Stakeholder Roles and Access-Level Mapping	62
A.3	Provisioner Actions and Required Roles	63
A.4	Ansible Roles and Host Targets	64
A.5	Test Layers	65
A.6	Resource-Use Scenarios	66

Glossary of Terms

admin daemon Host-local service on `greyfox-git` for operations requiring admin scope that the backend deliberately cannot perform: user deletion, sub-day membership expiry, custom hook installation, deadline scheduling.

Ansible facts Properties Ansible discovers about a host – addresses, operating system, hardware – before the first task of a play runs, readable by every later task and template.

Ansible inventory The list of target hosts and the groups they belong to. [HandIn](#) groups hosts both by stack (production) and by service, so shared roles can address a service group across stacks.

Ansible play The unit of an Ansible run that binds one inventory host group to the roles executed against it. [HandIn](#) has one play per production host: the GitLab, CI, and web plays, which must converge in that order.

Ansible playbook An ordered list of plays. `playbooks/site.yaml` imports the three host plays in dependency order and is the entry point for a full convergence.

Ansible role A reusable bundle of task files, templates, default variables, and handlers that a play applies to a host group. Roles are the unit of reuse and testing in the deployment repository.

Ansible vault Ansible’s mechanism for storing secrets encrypted at rest in the same repository as ordinary configuration, decrypted with a passphrase or key file at run time.

assignment image Per-assignment OCI image pushed to the mother project’s container registry. Bundles the compile/test toolchain plus the test artefact so student CI does not need to fetch them at job time.

assignment repository A single student’s copy of an assignment, seeded from the child template on claim, with access controlled according to the deadline lifecycle.

backend FastAPI service on `greyfox-web` that authenticates browsers via OAuth, enforces CSRF and request control, calls the GitLab API on the user’s behalf, and triggers the provisioner pipeline.

bootstrap state Runtime secrets generated by the GitLab Ansible play on first run: the OAuth application client ID/secret, the backend service PAT, the provisioner trigger token, runner registration tokens, and the system-hook secret. Consumed by the CI and web plays when they render environment files.

child template Per-semester copy of a mother template. Student assignment repositories are seeded from it when a student claims their invitation.

ci runner GitLab runner on **greyfox-ci** that accepts jobs tagged **ci**. Configured without host Docker socket access, so untrusted student/teacher code cannot escape its executor container onto the host.

CI variable GitLab CI/CD variable set at the group or project level. HandIn stores the group-level access-control config, assignment metadata, and per-mother claim/replay markers as CI variables rather than in a separate application database.

claim The lifecycle event of a student consuming an invitation token. The backend acquires a Redis lock, dispatches `INVITATION_ACTION=CLAIM_CREATE` (or `CLAIM_JOIN`) to the provisioner, and returns the resulting repository URL once the pipeline succeeds.

CLAIM_CREATE / CLAIM_JOIN / CLAIM_REJOIN The claim-related `INVITATION_ACTION` values dispatched to the provisioner. **CLAIM_CREATE** mints a new student assignment repository from the child template; **CLAIM_JOIN** adds the student to a shared group assignment; **CLAIM_REJOIN** restores Guest access after the deadline against a rejoin token and the mother's claim marker. **PREPARE** is the separate preflight and materialisation action.

claim marker CI/CD variable written into the mother project by the provisioner on a successful claim. Serves both as replay protection for further **CLAIM_CREATE** calls and as the ground truth for rejoin tokens.

custom hook A Git server-side hook installed on **greyfox-git** outside the repository, invoked by GitLab on push events. Installed by the admin daemon; used for repository-level policy that cannot be enforced through CI alone.

Docker executor GitLab runner executor mode that runs each job in an ephemeral Docker container. Both the **ci** and **provisioner** runners use it; the security difference is whether the host Docker socket is exposed to the container.

Docker socket The host's `/var/run/docker.sock`. Mounted only into the provisioner runner (needed for **docker build/docker push** of assignment images) and deliberately withheld from the **ci** runner so student code cannot escape its executor container.

frontend React/Vite single-page app served by Caddy. Pure view layer; never holds long-lived tokens or contacts GitLab directly.

greyfox-ci CI host. Runs the `ci` runner, the `provisioner` runner, and the Nexus dependency cache.

greyfox-git GitLab host. Runs GitLab CE, the container registry endpoint, and the admin daemon.

greyfox-web Public-facing web host. Runs Caddy, the React frontend, the FastAPI backend, and Redis.

HandIn The self-hosted programming-assignment platform this thesis describes: frontend, backend, provisioner, admin daemon, and the GitLab and CI hosts they run on.

idempotency key Client-supplied opaque string sent with a mutation request. The backend records the key in Redis with the request outcome so that a retried request is not re-executed and returns the original result.

invitation token HMAC-signed token minted by the backend that a teacher distributes to students. Encodes the assignment reference and (for test mode) a start-time gate and IP allowlist; consumed by the claim endpoint.

kaniko Google’s container-image builder that runs entirely in userspace. HandIn’s publish pipeline uses kaniko to `docker build` and `docker push` the assignment image from inside a CI job without requiring a privileged runner or the host Docker socket.

language template Cross-course starter that mother templates are created from. Defines the compile/test toolchain for one language (Java, Python, ...).

mother template Per-assignment root template repository owned by teachers. Holds the assignment configuration, hidden tests, and CI configuration that child templates and student repositories inherit.

Nexus Sonatype Nexus instance running on `greyfox-ci` that proxies Maven Central and PyPI, and exposes a Docker Hub proxy endpoint. Runner jobs route Maven and pip through it, while Docker pulls use it where explicitly configured, so common artefacts are not re-fetched over the internet on every run.

Postfix Mail transfer agent bundled with GitLab. Runs on `greyfox-git` and delivers outgoing GitLab mail through the filtering SMTP proxy hosted by the admin daemon.

proctor Optional teacher- or tutor-adjacent role used during test-mode assignments. Sees live claim state, start-time gate status, and IP-allowlist results for the exam session.

provisioner Privileged GitLab CI project that creates groups and repositories, manages memberships, runs invitations, and builds assignment images. Triggered by the backend over a signed pipeline trigger.

provisioner runner GitLab runner on `greyfox-ci` that accepts jobs tagged `provisioner`.
Has `host Docker socket access` because `assignment-image builds` must `docker build/docker push` into the mother project's registry.

rejoin token Project-scoped signed token that restores Guest access to an existing assignment repository after the deadline. Verified against the mother project's claim marker before Guest is granted.

request control Redis-backed package combining rate limiting, cooldowns, idempotency-key replay protection, and per-user distributed locks. Every mutation endpoint on the backend passes through it.

session cookie `HttpOnly + SameSite=Lax` cookie (Secure in production) issued by the backend after OAuth login. Identifies the browser session; the actual session state lives in Redis behind an opaque session ID.

system hook GitLab-wide shared-secret-protected HTTP callback that fires on events such as `user_create` or `project_create`. Used by the backend for the username-policy check.

test mode Gated access mode for exam-style assignments. Adds a start-time gate and an IP allowlist to the invitation token and enables a proctor view.

trigger token Long-lived GitLab pipeline trigger token attached to a specific CI project. The backend holds the provisioner's trigger token and uses it to start provisioner pipelines with `INVITATION_ACTION` / `ACTION` inputs.

List of Acronyms

ACME Automatic Certificate Management Environment (RFC 8555).

API Application Programming Interface.

CD Continuous Delivery.

CE Community Edition.

CI Continuous Integration.

CI/CD Continuous Integration/Continuous Delivery.

CSRF Cross-Site Request Forgery.

GHCR GitHub Container Registry.

HMAC Hash-based Message Authentication Code.

HTTP Hypertext Transfer Protocol.

HTTPS Hypertext Transfer Protocol Secure.

JAR Java Archive.

OAuth Open Authorization.

OCI Open Container Initiative.

PAT Personal Access Token.

PEM Privacy-Enhanced Mail.

PKI Public Key Infrastructure.

PyPI Python Package Index.

REST Representational State Transfer.

SMTP Simple Mail Transfer Protocol.

SPA Single-Page Application.

SSH Secure Shell.

TLS Transport Layer Security.

Chapter 1

Introduction

This thesis presents [HandIn](#), a self-hosted platform for managing programming assignments in university courses. An *assignment*, as the term is used here, is one self-contained programming exercise – a weekly exercise sheet, a lab task, an exam problem – that bundles the starter code students begin from, the tests their submission is checked against, the container image those tests run in, and the deadline after which write access ends. Teachers maintain it once as a [mother template](#); each semester derives a [child template](#) from it, and every student claims an [assignment repository](#) of their own that is seeded from that child. Students work in a browser-based workspace or with a local Git client, and can compile and test their current code from that workspace.

This structure is stored directly in a self-managed GitLab instance: courses, assignments, repositories, memberships, staff lists, and deadlines are GitLab groups, projects, and [CI variables](#) rather than rows in a separate database. In front of it sit a React [frontend](#) and a FastAPI [backend](#) that offer role-specific workflows instead of raw GitLab administration, while the privileged operations are delegated to a [provisioner](#) [Continuous Integration \(CI\)](#) project and an [admin daemon](#) on the GitLab host. Ansible converges the three production hosts – [greyfox-web](#), [greyfox-git](#), and [greyfox-ci](#) – and manages the credentials between them.

1.0.1 Motivation

Programming courses require a repeatable way to distribute starter code, collect submissions, run tests, enforce deadlines, and protect grading logic. Manual repository creation and manual membership management do not scale well once a course has many students, several tutors, repeated assignments, and tight deadlines. Mistakes in such a workflow are not merely inconvenient: a wrong permission can expose tests, a stale membership can extend a deadline unintentionally, and an inconsistent repository layout can make grading unreliable.

[HandIn](#) responds to these failure modes by taking manual GitLab administration out of the workflow. Every structural change – creating a course, handing out an assignment, granting access, letting it expire at the deadline – is carried out by the platform along a controlled

path instead of by a person editing GitLab state directly. Whether a permission is correct then becomes a property of the code, reviewable and fixable once, rather than of an action repeated by hand for every student and every semester.

1.0.2 Problem Statement

The technical tension is that fully automating GitLab-based assignment management pulls in two directions at once. On the one hand, the workflow must touch privileged GitLab state – creating groups, granting time-bounded memberships, writing [CI variables](#), installing [custom hooks](#) – which requires broad authority. On the other hand, the entity exposed to the public network is a web application that authenticates a browser over which it has no control. Any browser is treated as untrusted in this design: it runs on a machine the platform does not administer, the code it executes can be read and modified by whoever sits in front of it, and nothing arriving from it – a session cookie, a claimed role, a project path, a saved file – may be believed merely because the platform’s own [frontend](#) was supposed to have sent it. That web application must therefore decide every question of authorisation on the server, and it must not hold the broad authority the privileged half of the workflow requires. A naive design that gives the public backend an admin token collapses this tension; a design that leaves every privileged step to a human operator scales poorly and defeats the purpose. The platform must therefore *decompose* the authority: place enforcement at a single server-side chokepoint, delegate mutations to auditable executors (a provisioner [CI](#) project and an isolated admin daemon), and represent the *academic state* directly in GitLab objects rather than in a secondary database that would drift out of sync. By academic state this thesis means the facts used to administer a course: which departments and courses exist, which assignments belong to them, who teaches or tutors a course, which student holds which repository, and until when that student may write to it.

The central problem is to automate programming-assignment management while preserving four properties:

1. **Security:** untrusted student code and untrusted browser clients must not gain access to privileged credentials, hidden grading logic, or other students’ repositories.
2. **Reproducibility:** assignments, CI jobs, runtime environments, and deployment state must be reconstructable and auditable.
3. **Usability:** students, tutors, teachers, and administrators should interact with role-specific workflows instead of raw GitLab administration.
4. **Maintainability:** the platform has to stay operable by a small team for years, across many semesters of teaching. Recurring work – a new course, a new semester of an existing course, a new assignment, an additional language – must be a matter of configuration rather than of code changes, and the production hosts must be rebuildable from version-controlled Ansible instead of from manual state that has accumulated on them.

1.0.3 Thesis Scope

This thesis does not give every component equal depth. The [frontend](#), backend, provisioner scripts, CI templates, admin daemon, and Ansible roles all matter, but this thesis is structured around two deep focus areas:

- **Focus 1: Security and trust boundaries.** This includes authentication, sessions, CSRF protection, role checks, invitation tokens, [request control](#), username-policy enforcement, admin-daemon isolation, runner isolation, [Docker socket](#) separation, test-mode IP gating, and protection against leaked or over-scoped credentials.
- **Focus 2: Automated GitLab and infrastructure lifecycle.** This includes the GitLab namespace structure, provisioner pipeline architecture, course and assignment creation, invitation/claim flows, deadline handling, CI templates, per-assignment Docker images, package/registry integration, Ansible deployment, [bootstrap state](#), GitLab runner configuration, and integration tests.

1.0.4 Research Questions

1. How can a self-hosted assignment platform automate GitLab repository and membership management without exposing high-privilege GitLab credentials to the public web tier?
2. How can GitLab namespaces, CI pipelines, and repository files model the full assignment lifecycle without a separate application database?
3. How can Ansible and integration tests make a multi-host teaching platform reproducible and maintainable in production?

1.0.5 Contributions

The main contributions are:

1. A working self-hosted programming-assignment platform with a React workspace, a FastAPI backend, a GitLab-based state model, provisioner CI pipelines, and role-specific teacher, tutor, student, and admin workflows.
2. A security architecture that keeps privileged GitLab credentials out of the browser and out of the public backend write path, separates trusted and untrusted runners, and uses server-side enforcement for roles, requests, invitations, and test-mode access.
3. A GitLab namespace and provisioning model that represents departments, courses, template repositories, assignment groups, student repositories, CI variables, and policy files directly in GitLab.
4. An Ansible deployment that converges the GitLab host, CI host, and web host, rotates runtime credentials, renders backend configuration, configures runners, and supplies an executable integration-test role.

1.0.6 Structure of the Thesis

[Chapter 2](#) introduces the technologies used by the project with enough depth for the rest of the thesis to stand alone. [Chapter 3](#) derives stakeholder roles, functional and non-functional requirements, and the five design principles that follow from them. [Chapter 4](#) gives the system architecture across five production concerns, situates each component in the source-of-truth model, and closes with per-component implementation notes. [Chapter 5](#) is the first deep focus chapter, working from a threat model through credential separation, authentication, request control, invitation tokens, and the runner and admin-daemon boundaries. [Chapters 6 and 7](#) form the second deep focus area, covering the GitLab namespace, provisioner action surface, invitation and deadline lifecycles, and the Ansible plays that render the three-host deployment. [Chapter 8](#) discusses CI-based assessment, the private grader, per-assignment images, and caching. [Chapter 9](#) evaluates the system through unit and integration tests, a security-matrix review, and operational validation. [Chapters 10 and 11](#) close the thesis with trade-offs, limitations, and future work.

Chapter 2

Background

This chapter introduces the technologies [HandIn](#) is built on, at the depth needed to follow the design decisions in the remainder of the thesis. [Section 2.1](#) describes the GitLab objects the platform treats as its data model – groups, projects, memberships, CI/CD variables, and hooks – and [section 2.2](#) narrows this to what continuous integration means in the specific setting of graded programming assignments. [Section 2.3](#) covers the OAuth 2.0 Authorization Code flow and server-side session handling that separate a user’s identity from the credentials a public web application may hold. [Section 2.4](#) explains infrastructure as code with Ansible, [section 2.5](#) the container and runner isolation model that determines what untrusted student code can reach, and [section 2.6](#) the role of Redis as a store for short-lived coordination state. Readers already familiar with a given technology can safely skip the corresponding section.

One piece of vocabulary is needed first, because the sections below already have to say where a technology runs. [HandIn](#) is deployed on three servers, and the thesis refers to them by hostname throughout: [greyfox-web](#) carries the public web tier that browsers reach, [greyfox-git](#) runs the self-managed GitLab instance together with the privileged admin daemon beside it, and [greyfox-ci](#) runs the GitLab runners that execute student code, along with the dependency cache those jobs pull through. What each host contains in detail, and why the platform is split across three of them rather than one, is the subject of [section 4.2](#); here the names serve only as addresses.

2.1 GitLab as Code Host and Automation Platform

Git [6] is the distributed version control system in which all code in this thesis is stored: every clone of a repository carries its complete history, work happens locally, and changes are exchanged with a shared remote copy by pushing and pulling commits. GitLab [7] is a self-hostable platform built around such remotes, adding user accounts, access control, a web interface, a [Representational State Transfer \(REST\) Application Programming Interface \(API\)](#) served over [Hypertext Transfer Protocol Secure \(HTTPS\)](#), and an integrated CI system on top of the repositories it hosts. For this thesis it is therefore not only a Git hosting system: it

supplies the durable objects the platform is built on – groups, repositories, users, memberships, CI pipelines, package registries, container registries, and hook events – which is what makes it possible to hold the academic state of [section 1.0.2](#) in these objects instead of in a separate application database: a course is a group, an assignment is a subgroup, an enrolment is a membership, and a deadline is that membership’s expiry date.

Groups, subgroups, and projects. A GitLab *group* is a namespace that may contain projects and further subgroups. Subgroup nesting maps directly onto the platform’s namespace tree ([section 6.1](#)). A *project* is a Git repository plus its issue tracker, CI configuration, registries, and settings.

Memberships and access levels. Users are added to a group or project with one of five access levels: Guest (10), Reporter (20), Developer (30), Maintainer (40), or Owner (50). Access levels are inherited down subgroup and project trees; a Developer on a course group is a Developer on every assignment subgroup and student repository underneath. Memberships can carry an ISO-8601 `expires_at` date, but expiry is date-granular in GitLab [Community Edition \(CE\)](#), which is why the admin daemon adds sub-day scheduling ([section 6.7](#)).

CI/CD variables. GitLab [Continuous Integration/Continuous Delivery \(CI/CD\)](#) variables are key/value variables attached to groups or projects, optionally marked as protected or masked. Variables set on a parent group are visible to jobs in child projects. [HandIn](#) uses `HANDIN_*` variables on assignment groups to carry runtime state such as the assignment image tag, package coordinates, and language.

Trigger tokens and pipeline schedules. A project may register pipeline [trigger tokens](#) [12] that let an external service start a pipeline with a single [HTTPS](#) request carrying a chosen `ref`, action, and validated variable set. The backend uses one such token to fire the provisioner pipeline. GitLab also supports *pipeline schedules* that run a project’s CI on a cron expression; the provisioner registers such a schedule on itself when an assignment deadline is set, so that the post-deadline test run fires without an external trigger ([section 6.7](#)).

System hooks. GitLab instances can install [system hooks](#) [11], shared-secret-protected [Hypertext Transfer Protocol \(HTTP\)](#) callbacks that fire on events such as `user_create` or `project_create`. [HandIn](#)’s username-policy enforcement rides on the `user_create` hook, whose `X-Gitlab-Token` value is verified against a shared secret ([section 5.10](#)).

Package and container registries. GitLab CE bundles a per-project Maven, [Python Package Index \(PyPI\)](#), and generic package registry, and a per-project [Open Container Initiative \(OCI\)](#) container registry. [HandIn](#) stores compile-test artefacts (the test [Java Archive \(JAR\)](#) or wheel) in the mother project’s package registry and [assignment images](#) in its container registry.

Job tokens and job-token allowlists. Every CI job receives a short-lived `CI_JOB_TOKEN` [8] that other GitLab projects may accept under a configurable *job-token allowlist*. This is how a student’s compile job authenticates to the private grader project (see [section 8.3](#)) without ever seeing a long-lived secret.

2.2 Continuous Integration for Programming Assignments

Continuous-integration systems fit compile/test grading naturally. Each push produces a pipeline, each pipeline runs stages of *jobs* in declared containers, and each job emits status, logs, and typed artefacts. For a graded assignment this maps to two obvious jobs: compile and test. Job status becomes the grade signal, log output becomes the feedback the student sees, and artefacts (JUnit XML reports, pytest results, coverage files) become the structured basis for a downstream grader. Because pipelines are attached to specific commits, the mapping from “what the student submitted” to “how it ran” is a property of the CI system itself, not something the platform has to record separately.

The risk that comes with this convenience is that the graded body of code is untrusted. A student compile job runs code that the student wrote, inside a container that the platform provides on shared runner hosts. Three failure modes matter. First, code that reaches out over the network can exfiltrate other students’ work or hidden test material if the runner can see it. Second, code that shells out with privileges (for example via a mounted Docker socket) can escape the executor onto the host. Third, code that consumes unbounded CPU, RAM, or disk starves neighbouring jobs. The mitigations used in this thesis – Docker executor without the host socket, tmpfs-backed temporary directories, per-job resource limits, per-runner pull policies, and a strict split between the trusted [provisioner runner](#) and the untrusted [ci runner](#) – follow directly from those three failure modes.

2.3 OAuth 2.0 and Server-Side Sessions

The relevant [Open Authorization \(OAuth\)](#) 2.0 profile for this project is the *authorisation-code flow* [14, §4.1]. The browser is redirected to the identity provider (in this deployment, the self-hosted GitLab instance) with the client ID, requested scope, redirect URI, and a per-login random `state` parameter. The provider authenticates the user, requests consent for the scope, and redirects the browser back to the client’s redirect URI with an opaque *authorisation code* and the same `state` value. The client then exchanges the code for an access token *directly*, from server to server, with the client secret in the request body.

Two properties of this flow are load-bearing for the security model. First, the access token is never present in the browser: it is received server-side, stored server-side, and used server-side to call the GitLab API. Only an opaque session identifier reaches the browser as a cookie. Second, the `state` parameter binds a returning callback to the original login attempt, foreclosing a whole class of CSRF-on-login and mix-up attacks. Current guidance is the OAuth 2.0 Security

Best Current Practice draft [16], which the implementation broadly follows: authorisation-code with server-side exchange, an unpredictable per-login `state` value, exact redirect URI matching, and no implicit-flow tokens in the browser.

Server-side sessions materialise this decision. The backend (`backend/app/auth/router.py`) exchanges the code, stores the GitLab access token in Redis alongside the user record, and returns only an `HttpOnly`, `SameSite=Lax`, `Secure` session cookie. A companion, non-`HttpOnly` CSRF cookie carries a token that the browser reads and echoes back in a header for every state-changing request; the backend then compares the header, the cookie, and the session's stored value with constant-time equality before proceeding (section 5.3). This double-submit-cookie pattern with a strict origin check is the OWASP-recommended defence against cross-site request forgery [20, 21].

2.4 Infrastructure as Code with Ansible

Ansible [1] expresses infrastructure as YAML modules that describe a desired state and are executed over [Secure Shell \(SSH\)](#). An [inventory](#) lists the target hosts, optionally grouping them by function. *Group variables* attach configuration to those groups so that a single value (for example, `base_url`) can be reused by every role that touches the group. A [playbook](#) is an ordered list of [plays](#), each of which binds one host group to the [roles](#) that must run on it; a role packages task files, *templates* rendered per host, defaults, and handlers, and is the unit of reuse and testing. What a play knows about a host without being told – its addresses, operating system, and hardware – are Ansible [facts](#), gathered before the first task runs and readable by every task and template afterwards.

Idempotent convergence is the property that each task, when re-run against a host, either finds the desired state already in place (and reports *unchanged*) or converts the host to that state (and reports *changed*). For [HandIn](#), this means a re-run of the deployment does not, for example, create a second OAuth application or reset an already-rotated [Personal Access Token \(PAT\)](#).

The Ansible [vault](#) lets sensitive values live in the same repository as non-sensitive configuration by encrypting them at rest with a passphrase or key file. Secrets that only exist at rest in the vault (the GitLab root password and bootstrap token, the registry pull credentials, the Nexus admin password, the backend's invitation-signing secret, and the grader trigger token) travel through Ansible into the target host without ever appearing in plain text on disk under version control.

[HandIn](#) additionally needs *generated bootstrap state*: certain credentials are not decided offline but are minted by the GitLab play at first run (OAuth client secret, backend service PAT, provisioner trigger token, system-hook secret, runner registration tokens). Those values are then required by the later CI and web plays, which render them into the runner `config.toml`, the backend environment file, and the system-hook secret file, respectively. The bootstrap

state is therefore persisted on the GitLab host after the GitLab play, and the site playbook enforces the order `gitlab.yaml` $\rightarrow$ `ci.yaml` $\rightarrow$ `web.yaml` so that consumers only run after their producer has converged ([section 7.2](#)).

2.5 Containerisation and Runner Isolation

Containerisation appears in two distinct roles in this platform. On the service side, Docker [3] runs the long-lived processes with declared volumes, networks, and restart policies: the frontend and backend images, Redis, [Nexus](#) [24], and the admin daemon are Compose projects, while GitLab CE is a single Ansible-managed container. Caddy [2] is the exception – it runs as a host service from the distribution package rather than in a container, so that it keeps serving during a Compose restart. On the job side, GitLab Runner’s *Docker executor* [10] spawns a fresh container per CI job from a declared image, mounts the checkout into it, executes the job’s script, and destroys the container on completion.

The Docker socket, `/var/run/docker.sock`, is the primary risk surface for the job side [4]. A container that can talk to the socket can launch further containers on the host, bind arbitrary host paths, and therefore trivially escape its own executor. HandIn’s provisioner runner mounts the socket because it needs to build assignment images; the untrusted `ci` runner deliberately does not ([section 5.8](#)).

Runners bound student jobs with three additional mechanisms. *Resource limits* cap per-job CPU shares and memory (rendered from `gitlab_runner_docker_cpus_by_runner` and `gitlab_runner_docker_memory_by_runner`), and cap the number of processes with a `pids` limit. *Tmpfs mounts* back `/tmp` and `/var/tmp` with RAM-based file systems sized in megabytes; the job build directory remains on the runner build volume because current GitLab Runner releases already mount `/builds` and an additional `tmpfs` there creates a duplicate mount point. *Pull policy* governs how images are refreshed: the CI runner uses `[always, if-not-present]` so teacher-published image updates are checked first and a cached image remains usable during a transient registry outage, while the provisioner runner uses `[always]`.

The Nexus proxies on [greyfox-ci](#) act as a dependency cache. Maven and PyPI artefacts pass through Nexus once and are then served from local storage. Nexus also exposes a Docker Hub proxy endpoint used by explicit registry-mirror configuration, but ordinary Docker image pulls are not universally rewritten through Nexus.

2.6 Redis for Transient Coordination

Redis [23] is used in this platform strictly for state whose useful lifetime is measured in seconds to hours: OAuth sessions, CSRF tokens, sliding per-user and per-IP rate-limit counters, per-endpoint cooldown timers, idempotency records that de-duplicate retried mutations, and short-lived distributed locks that guard racy operations such as concurrent invitation claims. Each of these has three properties that make it a bad fit for GitLab repository state. First, it

is transient: a session that expires is uninteresting after the fact, and writing it to a Git object would only produce noise in the history. Second, it is high-write: rate limiters and idempotency records are updated many times per request, and a Git-backed store would multiply cost by a factor of thousands. Third, it is authoritative for a single decision now: locks and cooldowns must give a definitive answer without waiting for a merge-request round trip.

Redis matches these properties directly. Keys carry TTLs so entries expire without a sweep, atomic operations (`INCR`, `SET NX PX`) give race-free rate limiters and locks, and the memory footprint stays small because nothing is kept beyond its TTL. The GitLab repository state, in contrast, holds academic facts that should survive a Redis reboot: which student holds which repository, which assignment has which deadline, which teachers are on the roster. The design principles derived in the next chapter make this split explicit: GitLab owns durable academic state, Ansible owns infrastructure state, and Redis owns transient request state.

Chapter 3

Requirements and Design Principles

This chapter states what the platform has to do before the following chapters describe how it does it. It first identifies the stakeholders and the capabilities each of them needs, since almost every later access-control decision traces back to one of these roles, and then walks through the use cases those stakeholders drive end to end. From these it derives the functional and non-functional requirements, the latter capturing the security, reproducibility, and inspectability constraints that shape the architecture more strongly than any individual feature. Finally, it condenses these requirements into five design principles; they are referenced throughout the rest of the thesis as the justification for otherwise unobvious choices, such as running privileged GitLab writes outside the public backend.

Three terms recur in what follows. An *assignment* is the unit introduced in [chapter 1](#): one exercise together with its starter code, its tests, the container image they run in, and its deadline. Every assignment is either *individual*, giving each student a repository of their own, or a *team* assignment, in which a teacher-configured number of students – at most four – share one repository: the first of them to claim the invitation creates that repository, and the others join it by picking it from the teams that already exist in the same assignment. Finally, every assignment is written for one *language toolchain*. Java with Maven and Python are the two implemented paths, and each of them contributes its own project template, CI include file, and container base image, so that supporting a further language means adding those three artefacts rather than changing the platform.

3.0.1 Stakeholders and Roles

- **Student.** Claims assignments, edits files, saves submissions, runs compile/test jobs, and later regains read-only access if supported.
- **Tutor.** Supports students, monitors assignments, and may run compile/test flows, but must not manage course structure or create assignments.
- **Teacher.** Creates [mother templates](#), creates [child templates](#), configures assignments, publishes tests, manages tutors, and monitors progress.

- **Administrator.** Bootstraps departments and courses, manages teacher access, runs operational workflows, and owns deployment-level configuration.
- **Operator.** Runs Ansible, manages certificates, handles GitLab/runner/web host operations, and performs production deployments.

3.0.2 Use Cases

The platform exists to support the nine workflows below. Each use case names the actor who starts it, what that actor wants to achieve, and the state the system is left in once it completes; together they follow a course through its whole life, from an empty namespace to expired memberships after the last deadline. The functional requirements in [section 3.0.3](#) are what these workflows demand of the system.

1. **Open a course (Administrator).** An administrator registers a department and a course, names its teachers, and leaves behind a namespace in which those teachers can work without further administrator involvement.
2. **Author an assignment (Teacher).** A teacher creates a [mother template](#) from the project template of the language the exercise is written in, then fills it with the task, the starter code, the reference solution, and the tests, and configures deadline, team size, and access policy.
3. **Hand an assignment out (Teacher).** For the current semester, a teacher derives a [child template](#) from the mother, publishes the tests, and obtains a signed [invitation token](#) to distribute to the course.
4. **Start working (Student).** A student follows the invitation link, authenticates, and receives an [assignment repository](#): their own for an individual assignment, or a new or existing team repository for a team assignment.
5. **Solve and check (Student).** A student edits the solution in the browser workspace or clones it and works locally, and runs compile and test whenever they want, reading the resulting log to see what still fails.
6. **Supervise a cohort (Tutor, Teacher).** A tutor or teacher reviews how far the course has got, opens an individual student's repository, and reproduces a failing run without needing GitLab administrator rights.
7. **Close an assignment (System).** At the configured deadline the platform ends write access for every participant, so that what is in the repository at that moment is what gets assessed.
8. **Look back at submitted work (Student).** After the deadline a student regains read-only access to their own repository, without being able to change what was submitted.

9. **Hold an exam (Teacher, Proctor).** A teacher runs an assignment in test mode, restricting claims and workspace access to the network of the examination room, while a proctor follows the candidates' progress live.

Two further use cases carry no student-facing workflow but decide whether the platform can be run at all: an **operator** converges the three hosts from Ansible to deploy or rebuild the system, and an **administrator** verifies afterwards that the deployment behaves as intended.

3.0.3 Functional Requirements

1. Course, department, and assignment namespaces can be provisioned automatically.
2. Teachers can create mother templates from language-specific project templates; Java with Maven and Python are supported, and a further language is added as a template, a CI include file, and a base image rather than as a change to the platform.
3. Teachers can create child templates and signed invitation links.
4. Students can claim invitations and receive individual repositories, or – for team assignments – create or join a repository shared with up to three teammates.
5. Students can work in the web workspace or locally with Git.
6. Compile and test actions run through GitLab CI and return useful status and logs.
7. Deadlines are reflected in GitLab memberships and post-deadline workflows.
8. Teachers, tutors, and administrators have role-specific dashboards.
9. Test-mode assignments can restrict claims and workspace access to configured networks, and let a proctor observe candidates while they work.

3.0.4 Non-Functional Requirements

Six goals apply to the platform as a whole rather than to any individual feature. They carry the properties named in [section 1.0.2](#) into the design, and every architectural decision in the following chapters is justified against one of them.

1. **Security.** Untrusted student code and untrusted browsers must not be able to reach privileged credentials, grading logic, or another participant's repository, and every authorisation decision must be taken on the server. Where a decision is ambiguous, the system must fail closed rather than grant access.
2. **Self-hosting.** The platform must run entirely on hardware the department controls. Student code, submissions, identities, and assessment results must not leave that infrastructure, and no part of the workflow may depend on an external hosted service being available or trustworthy.

3. **Automation.** Normal operation – opening a course, handing an assignment out, granting and expiring access, assessing a submission – must not require anyone to administer GitLab or the hosts by hand. Manual intervention is reserved for exceptional cases.
4. **Reproducibility.** Assignments, CI runs, runtime environments, and the deployment itself must be reconstructable from versioned inputs, so that an assessment can still be explained after the fact and a host can be rebuilt from scratch.
5. **Maintainability.** A small team must be able to operate and extend the platform across many semesters, and the effect of a change must be visible in version control rather than only in the state of a running machine.
6. **Usability.** Students, tutors, teachers, and administrators must be able to do their work through workflows shaped to their role, without knowing how GitLab is administered underneath.

These goals impose the concrete constraints the rest of the thesis refers to directly:

1. The public backend must not hold a credential that can directly delete or rewrite arbitrary GitLab state.
2. Privileged GitLab writes must be auditable and constrained to explicit provisioner actions.
3. Student CI jobs must be isolated from privileged runner capabilities.
4. The deployment must be reproducible from Ansible, including generated secrets and GitLab bootstrap state.
5. Academic state should be inspectable in GitLab rather than hidden inside an opaque database.
6. The system must remain usable when teachers are not GitLab administrators.

3.0.5 Derived Design Principles

1. **GitLab is the durable academic state store.** Repositories, memberships, CI variables, schedules, and [claim markers](#) live in GitLab.
2. **The backend is the request gatekeeper.** It authenticates users, validates roles, enforces CSRF and request control, and decides which GitLab or provisioner action is allowed.
3. **Privileged writes are delegated.** Most GitLab mutations run inside provisioner CI pipelines; admin-only operations are isolated in the admin daemon on the GitLab host.
4. **Human policy and generated runtime state are separated.** Human-editable policy belongs in repository files such as `.handin/assignment.toml`; generated bindings and replay markers belong in CI/CD variables or daemon state.

5. **Trust levels must be visible in deployment configuration.** The trusted provisioner runner and untrusted student CI runner are separate Ansible-managed entities.

Chapter 4

System Architecture

This chapter describes the structure of the deployed system, moving from the outside in. [Section 4.1](#) names the production concerns and their relationships, [section 4.2](#) maps them onto the three production hosts, and [section 4.3](#) opens each host to show the processes, runners, and network paths inside it. [Section 4.4](#) then re-describes the same system in logical terms, independent of where each part runs, and [section 4.5](#) states for every kind of state which component owns it – the question that determines what the backend is allowed to do on its own. [Section 4.6](#) follows the main user-facing flows through these components, and [section 4.7](#) records per-component implementation notes. The two focus chapters that follow deepen the security boundaries ([chapter 5](#)) and the assignment lifecycle ([chapter 6](#)) sketched here.

4.1 Overview

[HandIn](#) consists of five production concerns: the web frontend, the FastAPI backend, the self-managed GitLab instance, the provisioner CI project, and the Ansible deployment. Alongside these sits the [admin daemon](#), the second privileged executor, which holds the GitLab-administrator operations the backend deliberately does not ([section 5.7](#)). Further supporting components include Redis, Caddy, Nexus, CI templates, project templates, and the teacher/admin CLI scripts.

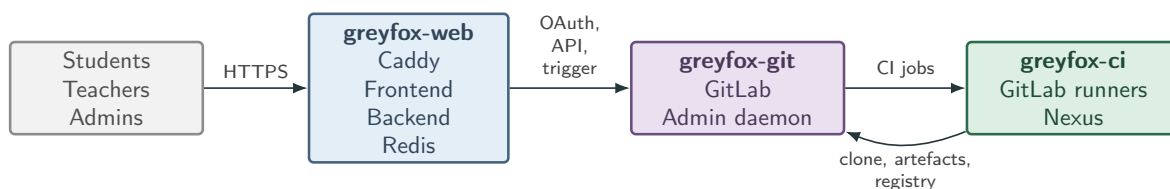


Figure 4.1: High-level production deployment.

4.2 Physical Deployment

The production deployment uses three hosts:

- **greyfox-web.** Public-facing web host. Runs Caddy, the React static frontend, the FastAPI backend, and Redis.
- **greyfox-git.** GitLab host. Runs GitLab CE, [Postfix](#), the GitLab Container Registry endpoint, a Caddy instance terminating TLS for the GitLab web UI and registry, and the **handin-admin-daemon** (which also hosts the filtering SMTP proxy that GitLab delivers its outgoing mail through).
- **greyfox-ci.** CI host. Runs GitLab runners, Nexus package proxies, and optional stress-test infrastructure.

4.3 Per-Host Internal Structure

The three-host overview in [figure 4.1](#) intentionally hides the internal structure of each host. The three diagrams in this section ([figures 4.2 to 4.4](#)) expand each host into its constituent processes and the arrows between them, so that the security and lifecycle chapters can refer to specific components rather than to hosts as opaque blobs.

The colour scheme is shared across the three diagrams. Orange outlines mark *privileged* components: the admin daemon, the provisioner runner, and privileged jobs. Green outlines mark stores and caches; blue outlines mark stateless services. Orange arrows mark connections that carry admin-only authority. Purple marks the GitLab surface.

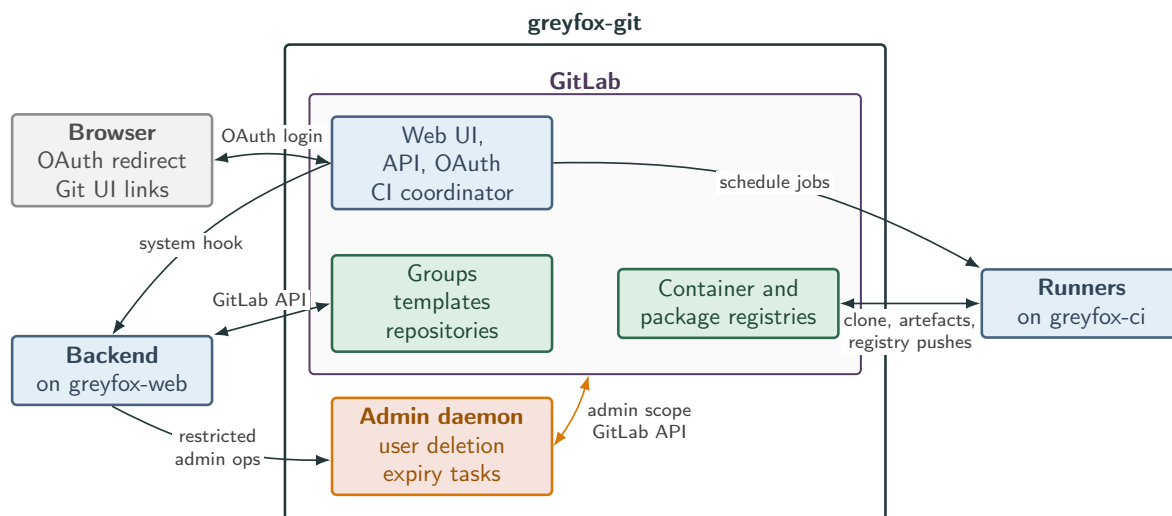


Figure 4.2: **greyfox-git** internal structure. GitLab is the persistent system of record; the admin daemon isolates the operations that need stronger privileges than the backend should hold.

Inside **greyfox-git** the GitLab process cluster hosts the web UI, REST [API](#), and OAuth 2.0 provider, and acts as the CI coordinator that schedules pipeline jobs onto the runners on **greyfox-ci**. The stores drawn inside the GitLab surface are the durable state it owns: department and course groups, mother and child templates, and student assignment repositories on one side, and the GitLab-managed OCI registry for per-assignment Docker images together

with the per-project Maven and PyPI package registries used for compile-test artefacts on the other. The admin daemon runs on the same host but is deliberately kept out of the GitLab process cluster; it owns the operations the backend must not be able to perform on its own – user deletion, precise sub-day membership expiry, custom hook installation, and deadline-scheduling state. The backend reaches it over a TLS-pinned bearer channel, and the daemon in turn talks to GitLab with admin scope, which is why the arrow between the two is drawn orange. The backend, runners, and browser appear only as context: the backend is expanded in [figure 4.4](#) and the runners in [figure 4.3](#), and the browser talks to GitLab directly only for the OAuth login redirect and the clickable git links.

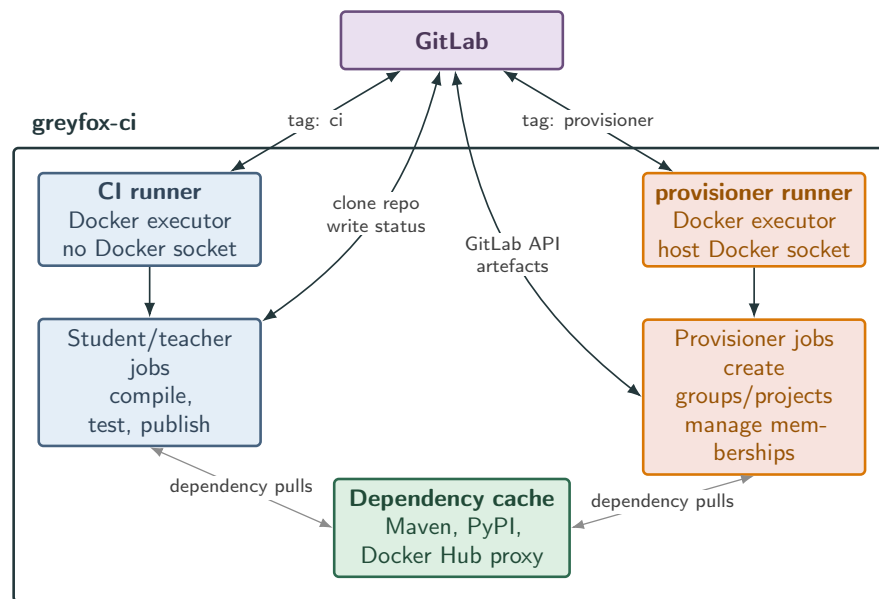


Figure 4.3: **greyfox-ci** internal structure. The trust distinction is runner-level: student jobs run without the host Docker socket; privileged provisioning is separated onto the provisioner runner.

GitLab is drawn above the **greyfox-ci** box because the runners always connect *up* to the CI coordinator: they pick up jobs and write status and artefacts back. Inside the host, the trust boundary runs between the two Docker-executor runners. The **ci** runner accepts jobs tagged **ci** and is configured without access to the host Docker socket, so untrusted student code running in an executor container cannot break out onto the host. The **provisioner** runner accepts jobs tagged **provisioner** and does have access to the socket, but only because assignment-image builds have to **docker build** and **docker push** into the mother project’s registry; nothing else on the CI host relies on it. Each runner produces one class of jobs. The student and teacher jobs – compile, test, and publish – run in the **ci** runner’s executor, clone their own repository, pull cached dependencies, and write status back to GitLab, and nothing else on the host is reachable from inside them. The provisioner jobs – create groups and projects, manage memberships, seed templates, update per-assignment CI variables – run in the privileged runner and authenticate to the GitLab API with the provisioner service token. Both classes route Maven and pip dependency downloads through the Nexus cache, and

Docker pulls use the Docker Hub proxy only where the runner or workload explicitly configures that mirror, so the same package artefact is not fetched over the internet on every run.

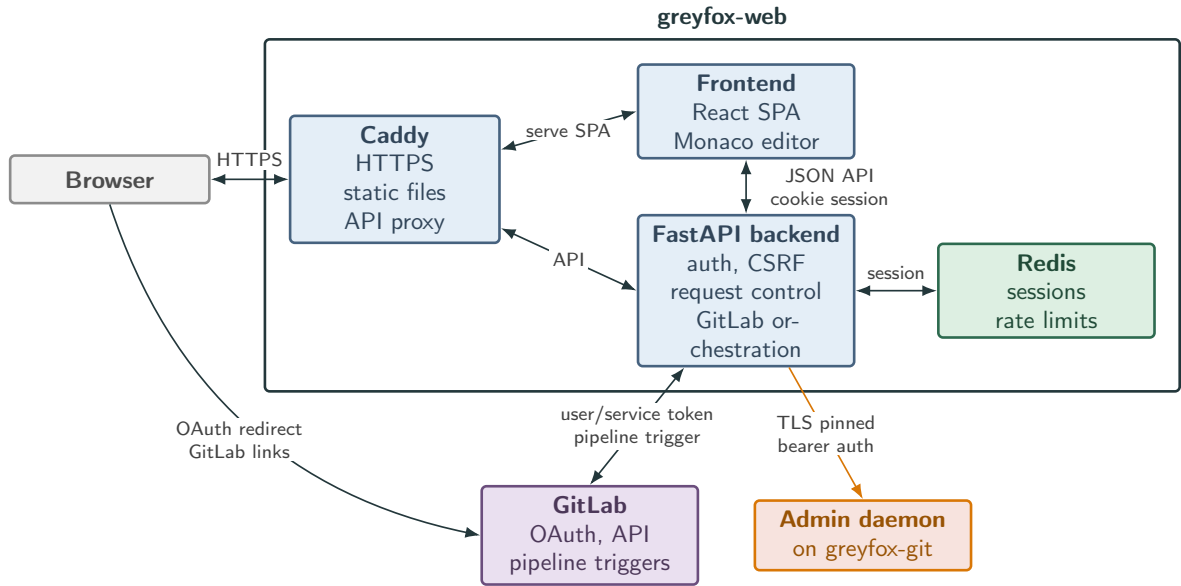


Figure 4.4: **greyfox-web** internal structure. The browser never receives service tokens. Privileged actions are validated in the backend, then expressed as GitLab API calls or provisioner pipeline triggers.

On **greyfox-web** the browser only ever talks to Caddy over **HTTPS**; the OAuth redirect to GitLab is the sole cross-host exception. Caddy terminates TLS, serves the static frontend bundle, and reverse-proxies **/api** to the backend, so nothing else on the host is directly exposed to the internet. The frontend itself is a React/Vite single-page app with a Monaco editor and is deliberately a pure view layer: it calls the backend JSON API with the session cookie and never holds long-lived tokens or contacts GitLab directly. The FastAPI backend is the single trust boundary for browser-driven requests. It owns OAuth session handling, CSRF, request control (rate limits, idempotency, cooldowns, and locks), and role and course authorisation, and it is the only component on this host that turns validated requests into GitLab API calls or provisioner-pipeline triggers. Redis backs its transient state – sessions, CSRF companion values, request-control counters, and short-lived locks. GitLab and the admin daemon are drawn outside **greyfox-web** to identify the two remote endpoints the backend reaches; the admin daemon is only ever contacted over the TLS-pinned bearer channel drawn orange.

4.4 Logical Components

Component	Responsibility
Frontend	React/Vite SPA with workspace, Monaco editor, project selector, CI panel, teacher/admin provisioning pages, tutor dashboard, invitation claim page, proctor view, and assignment config forms.

Component	Responsibility
Backend	FastAPI request gatekeeper. Owns OAuth session handling, CSRF checks, role enforcement, request control, GitLab read access, pipeline triggering, invitation tokens, workspace API, and admin-daemon client.
GitLab	Durable academic state: users, groups, projects, repositories, memberships, CI/CD variables, package registry, container registry, pipelines, schedules, and hook events.
Provisioner	Privileged GitLab CI project that creates and updates departments, courses, mother templates, child templates, student repositories, memberships, invitations, assignment images, and staff changes.
Admin daemon	Host-local service on greyfox-git for admin-only operations: user deletion, precise membership expiry, custom hook installation, deadline and grader scheduling state.
Ansible	Infrastructure source of truth for hosts, roles, firewall rules, GitLab bootstrap, runner configuration, deployment secrets, generated state, and integration tests.

4.5 Source-of-Truth Model

- **GitLab owns academic state:** repositories, groups, memberships, CI variables, assignment files, and pipeline history.
- **Ansible owns infrastructure state:** host roles, services, generated credentials, deployment topology, certificates paths, firewall policy, and runner configuration.
- **Redis owns transient request state:** sessions, CSRF data, request-control counters, idempotency records, cooldowns, and locks.
- **The backend owns decisions, not durable academic data:** it validates requests and triggers the correct GitLab/provisioner action.

4.6 User-Facing Workflows

The pieces described above are only useful in combination. The six workflows below are the paths that the rest of the thesis analyses in depth; here they establish the vocabulary and the actor sequence. Each step is a request from one role to one platform surface, so the workflows describe who talks to what before the finer security and lifecycle mechanics are introduced in [chapters 5 and 6](#).

1. Administrator creates a department and course, then adds teachers.
2. Teacher creates a mother template from a [language template](#).
3. Teacher creates a child template from the mother template, configures the assignment, and shares an invitation.
4. Student claims the invitation, receives a repository, edits code, saves, compiles, and tests.
5. Tutor or teacher monitors progress and handles test-mode/proctor workflows.
6. Deadline elapses, memberships expire, and post-deadline grading/test runs execute.

Every workflow above enters the platform through the frontend or CLI, crosses the backend as the single enforcement point, and settles in GitLab or the admin daemon as the durable state. This flatly excludes “users edit GitLab directly” from the supported paths, but the exclusion is about provisioning rather than authoring: no participant creates a repository, grants themselves access, or moves a deadline by editing GitLab, and the design assumes teachers and administrators are not GitLab administrators who should have to reason about GitLab paths, IDs, or CI variables. Writing the solution is the opposite case. A student’s [assignment repository](#) is an ordinary GitLab project on which the student holds Developer access until the deadline, so the [HandIn](#) workspace, a local Git client, and GitLab’s own web interface are equally supported and may be swapped at will. Only test mode withdraws that freedom, where a pre-receive hook installed by the admin daemon rejects any push that did not originate from the web editor, so that the network gate and proctor supervision cannot be sidestepped ([section 5.9](#)).

4.7 Implementation Notes by Component

Having placed each component in the source-of-truth model, this section records the concrete implementation choices per component. It closes the architecture chapter and establishes enough of each component’s internal shape to follow the security and lifecycle focus chapters that follow.

Backend. The backend is a typed FastAPI [\[22\]](#) application with routers for authentication, user/session information, groups, projects, provisioning, assignment configuration, staff, proctor operations, system hooks, and health checks. The single GitLab client and service layer avoid spreading HTTP details throughout the codebase.

Frontend. The frontend is a React 18 [\[17\]](#) application built with Vite. It embeds the Monaco code editor [\[18\]](#) for the in-browser workspace, and uses TanStack Query for server state, localStorage for drafts, and a typed API client that automatically attaches CSRF tokens to mutating requests.

Provisioner. The provisioner scripts are Python entry points invoked from GitLab CI. They validate environment variables, call the GitLab API, manage namespace creation, write files and variables, and emit artefacts that the backend can read.

Admin daemon. The admin daemon is a small FastAPI service with no public docs endpoint. It owns privileged GitLab-admin operations and persistent scheduling state.

Filtering SMTP proxy. The daemon also embeds an optional `aiosmtpd` listener that GitLab is repointed to for outgoing mail (`admin-daemon/handin_admin_daemon/smtp_proxy.py`). It exists to suppress a specific class of notification noise: when a tutor or professor is added to a course, GitLab emits one “you now have access to <repo>” membership mail per pre-existing assignment project – often hundreds at once. The proxy inspects each message’s **Subject** against a configurable pattern set (defaulting to GitLab’s *member access granted* wording), *drops* matches (accepting them at the SMTP layer with a 250 so GitLab does not retry), and forwards everything else unchanged to the host Postfix relay. The filter decision is a pure, unit-tested function that fails open – an empty pattern set or an undecodable message is forwarded, never silently discarded – so a misconfiguration cannot lose genuine mail. GitLab’s SMTP self-check is decoupled from its delivery port (it targets Postfix directly) because the proxy converges later than the GitLab role in the same play. The daemon’s own rejection mail continues to reach Postfix directly, bypassing the proxy.

CLI scripts. The teacher and admin CLI scripts exist as convenience wrappers, not as independent authority. They authenticate through the same OAuth flow as the browser (via a device-code-like callback), obtain a session cookie and CSRF token from the backend, and then call the same backend endpoints the [Single-Page Application \(SPA\)](#) calls. This is a deliberate design choice: if a teacher action is missing from the backend API, the CLI cannot implement it locally as a shortcut, because it has no independent path to GitLab. Adding the backend route first is the only way forward. The consequence is that every access control check in the backend applies uniformly to browser and CLI users.

Chapter 5

Focus 1: Security and Trust Boundaries

This is the first of the two focus chapters. It argues that the platform can be exposed to students and staff without granting the public web application broad authority over the GitLab instance behind it. [Section 5.1](#) fixes the adversary classes the design is evaluated against, and [section 5.2](#) shows how credentials are distributed so that no single compromised component holds enough authority to rewrite academic state. The next sections work through the enforcement points in the order a request meets them: authentication, sessions, and CSRF ([section 5.3](#)), server-side role and course authorisation ([section 5.4](#)), and rate limiting, cooldowns, idempotency, and locking ([section 5.5](#)). [Section 5.6](#) analyses the signed invitation and rejoin tokens, which are the only capability the platform hands to an unauthenticated party. [Sections 5.7](#) and [5.8](#) cover the two privileged executors and the boundary that keeps untrusted student CI jobs away from them, [section 5.9](#) the network restrictions used for supervised examinations, and [section 5.10](#) what all of this does and does not achieve against the threat model.

5.1 Threat Model

The system is evaluated against three adversary classes, chosen so that between them they cover the web-application risks catalogued in the OWASP Top 10 [\[19\]](#) that are reachable in this architecture – broken access control, cryptographic and identification failures, and security misconfiguration in particular:

1. an unauthenticated external attacker who can reach the public web endpoint;
2. an authenticated malicious student who can trigger their own workflows and run arbitrary code in their own CI jobs;
3. a compromised public backend process that should still not be able to destroy or arbitrarily rewrite GitLab state.

The assets in scope for the threat model are:

- **GitLab repositories.** All mother templates, child templates, and student repositories, including their commit history and access-control records.
- **Grader and test logic.** Reference solutions, test JARs/wheels published to the mother project's package registry, and the private grader project itself.
- **Student submissions.** Commits pushed by students to their own repositories, protected against cross-student read/write access.
- **Credentials.** OAuth client secret, backend service PAT, provisioner trigger token, provisioner bot PAT, admin-daemon PAT, admin-daemon bearer secret, invitation [Hash-based Message Authentication Code \(HMAC\)](#) secret, and grader trigger tokens.
- **Runner host.** The `greyfox-ci` machine, its Docker daemon, its persistent image cache, and the Nexus proxies it hosts.
- **Session cookies and CSRF cookies.** Long-lived enough to matter to a session-fixation attacker.
- **Invitation and rejoin tokens.** Signed values that carry assignment identifiers and (in [test mode](#)) time and IP metadata.
- **Deadline and expiry state.** Persistent scheduler entries in the admin daemon's state store and `expires_at` fields on GitLab memberships.
- **Audit trail.** GitLab pipeline logs, provisioner CI job artefacts, the admin daemon's structured logs, and system-hook records.

5.2 Credential Separation

The backend is a controlled gatekeeper, not the holder of broad GitLab authority. Its service token is scoped to read-oriented API access, while write operations are expressed as provisioner pipeline triggers or routed to the admin daemon. The provisioner holds its GitLab API token inside GitLab CI variables, and the admin daemon holds admin-only credentials on the GitLab host.

5.3 Authentication, Sessions, and CSRF

The system uses GitLab OAuth rather than a local password database. The backend exchanges the authorisation code server-side, stores the GitLab token in server-side session state, and returns only opaque cookies to the browser. State-changing requests require a CSRF token that matches both the readable CSRF cookie and the session value.

Credential	Location	Purpose
OAuth client secret	Backend environment rendered by Ansible	GitLab login flow.
Backend service PAT	Backend environment	Read-only GitLab API access, including pipeline status and artefacts.
Provisioner trigger token	Backend environment	Start provisioner pipelines only.
Provisioner bot PAT	Provisioner CI variable	GitLab mutations inside provisioner jobs.
Admin daemon PAT	Admin daemon environment on greyfox-git	Admin-only user deletion, forced membership removal, hooks, state operations.
Admin daemon bearer secret	Backend and daemon environments	Authenticates backend-to-daemon HTTPS calls.
Invitation secret	Backend environment	HMAC -signs invitation and rejoin tokens.

Table 5.1: Credential placement and intended authority.

5.4 Server-Side Role and Course Authorisation

The current implementation uses GitLab group membership on `<course>/template-repositories` as the canonical staff anchor:

- Owner access means teacher-level course management.
- Maintainer access means tutor-level course support.
- Administrators are derived from configured admin users or GitLab admin identity.

The frontend hides unavailable UI, but the security boundary is the backend role check.

5.5 Request Control

The Redis-backed request-control layer combines rate limiting, cooldowns, idempotency records, and distributed locks. This is security-relevant because invitation claims, repository creation, and provisioning actions are expensive and race-prone.

- Rate limits bound the load generated by abusive clients.
- Cooldowns prevent repeated expensive actions.
- Idempotency avoids duplicate repositories after retries.
- Locks prevent concurrent claims or mutations from corrupting shared state.
- The IP resolver trusts forwarded headers only from configured proxies.

5.6 Invitation and Rejoin Token Security

An invitation is the primary way a student ends up with a repository. The design pins two properties: an invitation is stateless enough to share as a URL, and no student, teacher, or observer can tamper with its contents or replay it against a different assignment. Both properties are met with an [HMAC](#)-signed compact payload rather than a database-backed token.

Token structure. The payload is a JSON object carrying: a version tag `v`; a `kind` of `join` or `rejoin`; the invitation ID; the assignment slug; the creating teacher’s user ID; the GitLab project and group IDs the token authorises against (mother project, teacher group, enrolment group, target namespace, student template project, target project); the template ref and file globs to seed the student repository; the group-member count that distinguishes individual from group assignments; the ISO-8601 deadline; and, for test-mode assignments, the start time, allowed IP ranges, and a `web_push_only` flag. The token is base64url-encoded and suffixed with an [HMAC-SHA-256](#) [5, 15] signature over the payload using the backend’s `invitation_secret`. The signature is verified in constant time (`hmac.compare_digest`).

Why signature and not opaque token lookup. The alternative would be to store one row per outstanding invitation and look it up on claim. This project chose signed tokens because it avoids a durable state item per invitation, lets the resolve endpoint work without a database write, and makes the token self-contained across a possible Redis flush. The trade-off is that the token cannot be shortened: it is roughly 1–2 kB, and `invitation_token_max_length` is capped at 8 kB defensively.

Replay markers. Signed tokens on their own cannot enforce “claim exactly once.” The provisioner therefore writes a per-claim marker into a CI/CD variable on the mother project when a `CLAIM_CREATE` succeeds. On a repeated claim the marker fires and the second attempt is either idempotent (same student, same repository) or rejected. Rejoin uses the same marker to bind a returning student back to the same repository rather than creating a second one.

Claim-create versus claim-join. `CLAIM_CREATE` is issued when the student is the first to claim a group assignment or the sole claimant of an individual assignment; it *creates or reuses* the student repository from the already prepared child template. `CLAIM_JOIN` is issued when the student joins an existing group [assignment repository](#) that a teammate has already claim-created. The invitation payload’s `group_member_count` field decides which path applies: `supports_join` is true when `kind == "join"` and `group_member_count > 1`.

Post-deadline rejoin. After the deadline, initial memberships expire ([section 6.7](#)). To let students re-view their work, the backend can mint a [rejoin token](#) bound to the mother project and the claim marker; `CLAIM_REJOIN` then re-adds the student as a Guest to their own repository. The rejoin path never grants Developer, so the deadline still stops writes.

Idempotency and locking. Each mutation endpoint composes rate limiting, per-endpoint cooldown, a Redis idempotency record keyed on the `Idempotency-Key` header, and a short-lived distributed lock before triggering the provisioner. Two racing claims cannot both fire `CLAIM_CREATE`: the lock serialises them, the idempotency record short-circuits the second, and the marker forecloses the third.

5.7 Admin Daemon Isolation

The admin daemon is the boundary for operations that require GitLab administrator privileges. It runs on the GitLab host, exposes no public documentation endpoints, uses TLS, requires a bearer secret, and is additionally restricted through firewall/source-IP policy. This lets the backend request user deletion, precise expiry, custom hook installation, and deadline-triggered jobs without receiving the admin PAT.

5.8 Runner Trust Boundary

The CI host runs different runner configurations for different trust levels. Provisioner jobs are trusted operator-controlled jobs and may need Docker host access for image operations. Student compile/test jobs execute untrusted code and must not receive the host Docker socket.

- **provisioner runner.** Handles privileged provisioning and image-build jobs. It can mount the Docker socket when needed.
- **ci runner.** Handles student, teacher, tutor, and grader compile/test jobs. It does not mount `/var/run/docker.sock`; it also uses resource caps and a cache/pull policy tuned for repeated short jobs.

5.9 Test-Mode Security

Test-mode assignments add controls for exam-like settings: start-time gates, network allowlists, proctor bypass, web-push-only hooks, WIP branch autosave, submit semantics, and proctor supervision. The IP gate deliberately uses the socket client IP for test-mode access checks and does not trust arbitrary `X-Forwarded-For` values from students.

5.10 Security Evaluation

Threat	Defence in this system	Residual risk
Compromised backend process	Backend holds only a service PAT (read-oriented) and a provisioner trigger token; admin-only operations require the admin daemon, reachable only via bearer secret over TLS from configured source IPs.	A compromised backend can still read GitLab data available to its service PAT and can trigger any provisioner action the daemon accepts from it.
Malicious student CI job	ci runner has no Docker socket, uses tmpfs for temporary directories, has resource caps, and runs a fresh container per job.	Kernel- or Docker-engine vulnerabilities could still allow escape; a job may perform network-level abuse against reachable services.
Forged invitation token	Payload is HMAC -signed with a server-side secret and verified in constant time; replay is bounded by per-claim markers in mother CI variables.	Loss of <code>invitation_secret</code> would allow forgery; rotation invalidates all outstanding invitations.
Cross-student repository access	Repositories live under distinct namespaces; membership is granted only to the claimant (and teammates for group assignments). The backend does not proxy read access.	A misconfigured provisioner action that creates memberships too broadly would bypass this; mitigated by the enumerable action surface and integration test.
Spoofed X-Forwarded-For	IP resolver honours forwarded headers only when the peer is in <code>trusted_proxies</code> ; test-mode gate uses the socket IP, never the header.	Misconfigured <code>trusted_proxies</code> on a new deployment would reintroduce header trust; Ansible defaults are conservative.
Leaked provisioner trigger token	Token only starts a pipeline on a configured project; the provisioner validates <code>ACTION</code> and every input pattern before touching GitLab.	An attacker with the token could still trigger valid actions; rotate via Ansible if leaked.

Threat	Defence in this system	Residual risk
Stale membership past deadline	Developer membership is revoked by the admin daemon at the exact deadline timestamp; GitLab's day-granular <code>expires_at</code> remains as a fallback. Post-deadline access is Guest-only via rejoin.	Deadline updates against existing memberships and queued daemon entries require careful coordination; see section 6.7 .
Invalid username at signup	GitLab sends <code>user_create</code> events with the system-hook shared secret; the backend verifies the token and forwards to the admin daemon, which validates the username against a canonical regex and deletes non-exempt off-policy users.	If the regex is misconfigured to be too permissive, the check silently accepts bad usernames; if too strict, legitimate signups are deleted.

Chapter 6

Focus 2: GitLab-Based Assignment Lifecycle

This chapter follows a course from an empty GitLab namespace to expired student memberships after a deadline. [Section 6.1](#) establishes the namespace layout that serves as the platform’s data model, and [sections 6.2](#) and [6.3](#) draw the line between human-editable policy committed to repositories and generated state written by automation – a separation that later makes it possible to tell teacher intent from machine bookkeeping. [Section 6.4](#) introduces the provisioner pipeline as the executor of privileged writes and its action surface as the set of operations the backend may request at all. The remaining sections trace the lifecycle itself: course and assignment creation ([section 6.5](#)), the invitation claim protocol and its replay handling ([section 6.6](#)), the deadline and post-deadline rejoin path ([section 6.7](#)), and the workflows available to teachers, tutors, and proctors ([section 6.8](#)). [Chapter 7](#) then explains how Ansible creates the platform this lifecycle runs on.

6.1 Namespace Design

The GitLab namespace is the platform’s data model: every durable academic object – platform tooling, courses, staff rosters, mother templates, and student repositories – is a group or project at a well-defined path under a single root group, denoted `<university>` in what follows. Because authorisation, provisioning, and policy lookup all key off these paths, the namespace layout is a deliberate design artefact rather than an incidental side effect of usage.

The structure is easiest to read at three levels, each drawn as a rooted tree in [section A.1](#). The trees are conceptual: a real installation has many departments, courses, semesters, and student repositories, but the *shape* at each level is fixed.

- **Platform level** ([figure A.1](#)). Directly under `<university>` sit the platform-wide, course-independent projects. The `provisioner` CI project executes every privileged action and also versions `username-regex.txt`; the merged `ci-templates` repository holds

the per-language `/java-maven` and `/python` include files and the root `.gitlab-ci.yml` grader pipeline; the `stresstests` group collects per-assignment stress-test projects; and `project-templates` provides one language starter (`java-maven`, `python`) per supported toolchain. These exist once for the whole installation and are shared by every department and course.

- **Course level (figure A.2).** Each academic course lives at `<department>/<course>`. It contains a `course-master-repo` for course-level defaults, the `template-repositories` group that anchors staff authorisation and holds the mother templates, and one subgroup per `<semester>` in which the course was actually taught.
- **Assignment level (figure A.3).** Within a semester, each `<assignment>` is a subgroup holding the child `template` (created from a mother) and the per-student or per-team repositories forked from that template.

Two properties of this layout carry weight later in the thesis. First, the `template-repositories` group is the authorisation anchor for teaching staff (Owner = teacher, Maintainer = tutor), so it plays a dual role: it hosts the mother templates *and* it defines who may act as a teacher or tutor for the course (section 5.4). Second, the split between the platform level and the course/assignment levels mirrors the source-of-truth model of section 4.5: the platform-level projects are installed once and rarely change, whereas the course and assignment subtrees are materialised on demand by the provisioner as staff drive the assignment lifecycle.

6.2 Repository Files as Policy

Human-editable policy is versioned in GitLab repositories:

- `.handin/course.toml` in `course-master-repo` for course-level defaults.
- `.handin/assignment.toml` in each mother template for assignment metadata, image metadata, access settings, test-mode options, and deadline information.
- `username-regex.txt` in `<university>/provisioner` for the username policy consumed by the system-hook flow.

Earlier iterations of the platform used a per-course `teaching-staff` project holding `.handin/teachers.txt` and `.handin/tutors.txt` rosters, updated via merge requests. That flow has been retired: staff authorisation is now derived directly from GitLab group membership on `<course>/template-repositories` (see section 5.4), which removes a second source of truth and lets access-level changes take effect without a repository write.

6.3 Generated Runtime State

Generated state belongs in CI/CD variables or daemon-managed state:

- `HANDIN_*` variables on groups and projects for assignment runtime data.

- Claim/replay markers to prevent duplicate invitation claims.
- Grader/project/image references.
- Admin-daemon state for expirations, hooks, and deadline-triggered jobs.

6.4 Provisioner Pipeline Architecture

The provisioner is a GitLab CI project triggered by the backend with an action name and a set of validated variables. Its `scripts/dispatch.py` maps each **ACTION** to a runner, so the action surface is small and enumerable. The currently dispatched actions are:

- namespace and structure: `create_department`, `create_course`, `create_mother`, `create_child_template`, `create_student_repo`
- invitation flow: `PREPARE`, `CLAIM_CREATE`, `CLAIM_JOIN`, `CLAIM_REJOIN` (dispatched via a separate `invitation_dispatch`, keyed on `INVITATION_ACTION`)
- images and grading: `build_assignment_image`, `run_deadline_tests`
- scheduling and operations: `update_deadline_schedule`, `gather_due_date`, `lock_main`, `enforce_username_policy`, `run_stresstest`

Notably, there is no standalone `update_template` action: template updates reuse `create_child_template` in an update mode, which keeps the initial-create and update paths on a single code path.

For the **ACTION** surface, a dedicated validation stage runs `scripts/validate_inputs.py` before any GitLab call, checking paths, usernames, semesters, assignment names, globs, numbers, timestamps, and boolean values against strict patterns. The invitation pipeline skips that stage deliberately: its inputs are not free-form operator input but fields the backend has already validated while verifying the token's signature, and the dispatcher re-checks their presence and type on arrival.

6.5 Course and Assignment Creation

The following numbered protocol traces one complete lifecycle from an empty department to an invitable assignment. Each step names the actor, the platform surface it enters, and the resulting change in GitLab or the admin daemon.

1. Admin creates a department and course.
2. Teacher creates a mother repository from a language-specific template.
3. The provisioner writes assignment metadata and CI configuration.
4. Teacher publishes or verifies tests.
5. Teacher creates a child template for a semester, assignment name, and deadline.

6. The platform creates an invitation token for students.

Expanded step by step:

1. **Department and course.** Admin submits a form to the backend, which triggers `ACTION=create_department` and, once that finishes, `ACTION=create_course`. The provisioner creates `<department>/` and `<department>/<course>/` groups, the `course-master-repo` project, and the `template-repositories` subgroup that will act as the staff anchor.
2. **Mother repository.** Teacher chooses a language template (Java/Maven or Python). The backend triggers `ACTION=create_mother`. The provisioner clones the corresponding `<university>/project-templates/<lang>` project into `<course>/template-repositories/<mother>`, writes `.handin/assignment.toml` with default access globs and image metadata, and sets `HANDIN_*` variables on the mother project. It then triggers an initial `build_assignment_image` so child templates have a valid image reference from day one.
3. **Publish tests.** Teacher hits *Publish*. The backend triggers the mother project's CI with `inputs.action=publish`, which runs two jobs: one pushes the test JAR or wheel to the mother's package registry, the other builds an assignment image with [kaniko](#) [13], bakes the freshly built artefact into it, and pushes it to the mother's container registry [9]. Both jobs authenticate with the mother's own `CI_JOB_TOKEN`, so no external credentials are involved.
4. **Child template.** Teacher submits assignment name, semester, deadline, group size, and any test-mode options. Backend triggers `ACTION=create_child_template`. The provisioner creates the assignment subgroup under `<course>/<semester>/<assignment>/`, seeds the child template project from the mother, propagates `HANDIN_ASSIGNMENT_IMAGE` and `HANDIN_ASSIGNMENT_IMAGE_DIGEST`, and writes the deadline into the assignment's TOML and CI variables.
5. **Invitation token.** Teacher chooses individual or group assignment and clicks *Invite*. Backend mints an [HMAC](#)-signed token (see [section 5.6](#)) and returns the invitation URL to the teacher, who distributes it out-of-band.

6.6 Invitation Claim Lifecycle

A [claim](#) proceeds in three visible phases and one background phase. [Figure 6.1](#) maps these phases onto the actors and platform surfaces involved.

Resolve. The student opens the invitation URL. The backend decodes and verifies the token, applies the test-mode start-time gate and IP allowlist if the token is a test-mode invitation, and returns a sanitised summary (assignment slug, deadline, group-member count, whether joinable, whether repository already exists) without ever exposing GitLab IDs or the raw

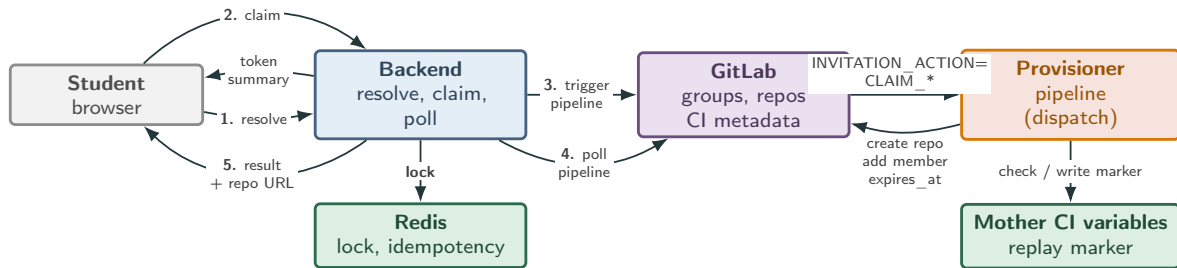


Figure 6.1: Invitation claim sequence. Steps are numbered in temporal order: (1) resolve verifies the [HMAC](#) and returns a sanitised summary; (2) the student confirms with an [idempotency key](#); (3) the backend, having acquired a Redis lock, triggers the provisioner; (4) the frontend polls; (5) success is reported with the new repository URL. The replay marker in mother CI variables forecloses a duplicate `CLAIM_CREATE`.

token to the browser. This endpoint is rate-limited independently of the claim endpoint so pre-start-time polling cannot exhaust the claim budget.

Claim. On confirm, the backend takes an idempotency key from the browser, acquires a per-user distributed lock, dispatches `INVITATION_ACTION=CLAIM_CREATE` or `CLAIM_JOIN` and returns the provisioner pipeline ID. Which action applies is decided by the backend from the token payload before the pipeline is triggered; the provisioner receives the already-validated fields as individual pipeline variables rather than the token itself. The provisioner `invitation_dispatch.py` then checks the mother project’s replay marker to reject an already-consumed claim, creates or reuses the student repository inside the assignment subgroup that `create_child_template` materialised earlier, seeds it from the child template, adds the student as a Developer with `expires_at` set to the day after the deadline, and writes the claim marker.

Pipeline polling and result artefact. The frontend polls the pipeline endpoint with the returned pipeline ID until the pipeline terminates. On success the provisioner emits a JSON *result artefact* on the job – `artifacts/invitation_result.json` for all four invitation actions, and a per-action file such as `artifacts/create_child_template.json` for the teacher-driven `ACTION` surface – which carries the action, the created project’s ID, and its path and web URL. The backend surfaces this to the frontend on the poll response.

Membership expiry (background). The initial Developer membership carries GitLab’s day-granular `expires_at`, set to the day after the deadline, as a fallback. In parallel, the admin daemon schedules exact membership revocation for the deadline timestamp and fires the post-deadline test run at the same point (section 6.7). Once the membership is revoked, the repository remains present but the student can no longer push. If the assignment supports post-deadline review, the backend mints a `CLAIM_REJOIN` token bound to the mother project and the claim marker, which restores Guest access on demand.

6.7 Deadline Lifecycle

Deadlines affect GitLab memberships, assignment metadata, precise expiry scheduling, and post-deadline assessment. GitLab’s built-in membership expiry is date-granular, so the platform adds daemon-side scheduling for more precise revocation and deadline-triggered work.

The admin daemon holds a `StateStore` (`admin-daemon/handin_admin_daemon/state.py`) whose canonical form is a JSON file persisted in a dedicated GitLab repository on `greyfox-git`. Each pending `Expiration` record carries a kind, a target kind and ID (group or project), the affected user, the exact Unix timestamp to fire, and an idempotency key. The kind selects the handler: `deadline_test_run` triggers the post-deadline test pipelines, and every other kind (`tutor` for staff expiry, `claim` for a student membership) falls through to the default handler, which revokes the group or project membership. Custom hooks are not scheduled at all – they are separate `CustomHook` records in the same state file, installed immediately on request. An in-memory heap gives the scheduler $O(\log n)$ access to the next due entry; the persistent file gives crash-safety.

The `ExpiryScheduler` runs one asyncio task that peeks the heap, sleeps until the next deadline (or until `kick()` fires sooner), and then dispatches to GitLab. A separate resync task periodically reloads the state file so out-of-band edits (an operator adding a deadline directly) are picked up without a daemon restart.

The two mechanisms divide the work as follows. GitLab’s `expires_at` field on a student’s Developer membership is set to the day after the deadline and serves as a day-granular fallback. Exact deadline enforcement is handled by the admin daemon: when a student claims an assignment, a corresponding expiration is scheduled for the precise deadline timestamp. At that instant, the scheduler revokes the student’s Developer membership and triggers the post-deadline test run. The daemon therefore supplies the sub-day precision that GitLab CE’s built-in membership expiry lacks, while `expires_at` provides an additional fallback at the GitLab level.

The grader trigger token never travels through a request body: it lives in the daemon’s env file (rendered by Ansible from vault) so a leaked schedule payload does not leak a token.

A remaining limitation is deadline *updates*. Changing the deadline after invitations have been claimed requires re-issuing GitLab `expires_at` on every existing membership and updating (or deleting) any queued daemon entry keyed by the old deadline. The platform detects the case and schedules the fix, but it cannot retroactively re-add students whose memberships have already been removed at the old deadline; those students go through the rejoin path.

6.8 Teacher, Tutor, and Proctor Workflows

Staff panel. Teachers manage tutors through the staff panel on `TeacherProvisionPage.tsx`. Adding a tutor calls the backend, which validates the caller is Owner on the course’s

`template-repositories` group, then triggers a provisioner action that adds the tutor as Maintainer on that group and, transitively via the provisioner's reconciler, on every active assignment subgroup so tutors can run `test/compile` pipelines against student repositories.

Tutor dashboard. Tutors get a read-focused dashboard listing active assignments in the courses they support, with drill-down into student progress (submission timestamps, pipeline status, saved files). Because tutors are Maintainer rather than Owner, they cannot invoke `create-assignment` or `invite` endpoints; the backend's role check returns the `tutor` role tag, which the API layer maps to a read-only surface for those routes.

Proctor supervision. For test-mode assignments the platform adds a proctor view that subscribes to student workspace autosaves in near-real-time. Proctors are tutors or teachers whose IP is in the classroom allowlist; the backend enforces the allowlist server-side and gates the proctor endpoint behind the same course-authorisation check.

Cache invalidation. Course authorisation is cached in Redis with a short TTL (default 30s, per the docstring of `course_authorization.py`). Explicit invalidations fire on `add-tutor`, `remove-tutor`, and `role-change` operations so the newly added tutor immediately sees the courses they now support.

Why the tutor/teacher split matters. A tutor is a person who helps students but should not silently change the shape of the course. Elevating a tutor to teacher requires an existing teacher's explicit action; elevating a teacher requires the admin. This mirrors the GitLab access-level ladder and makes accidental privilege escalation visible in the group's membership audit log.

Chapter 7

Deployment and Operations with Ansible

This chapter describes how the three-host production stack is created and kept reproducible from code; the Ansible vocabulary it uses throughout – [inventory](#), [play](#), [role](#), [facts](#), and [vault](#) – is introduced in [section 2.4](#). [Section 7.1](#) presents the inventory model that separates hosts by stack and by service, and [section 7.2](#) explains why the [plays](#) must converge in a fixed order: the GitLab play generates the credentials the other two consume, so running it alone leaves the rest of the stack holding stale secrets. [Sections 7.3](#) to [7.5](#) then walk through the three plays in that order, covering what each one installs, which secrets it generates or consumes, and how the trusted provisioner runner is kept distinct from the untrusted student runner. [Section 7.6](#) covers the firewall, TLS, and network boundaries enforced around these hosts, and [section 7.7](#) the image build and deployment pipeline that delivers new application versions onto the web host.

7.1 Inventory Model

The Ansible inventory groups hosts by stack and by service. In the production stack, `<university>_prod_gitlab`, `<university>_prod_ci`, and `<university>_prod_web` map to the three hosts, and also belong to service groups used by shared roles.

7.2 Playbook Ordering and Bootstrap State

The site playbook imports the GitLab, CI, and web plays in dependency order. This order matters because the GitLab play creates or rotates OAuth credentials, PATs, trigger tokens, runner credentials, registry data, and system-hook secrets. The generated bootstrap state is then consumed by the CI and web plays.

```
gitlab.yaml -> ci.yaml -> web.yaml
```

7.3 GitLab Play

The GitLab play converges `greyfox-git` in a fixed sequence implemented across the `roles/gitlab/tasks` files, imported in the order `gitlab.yml` (which in turn imports `wait.yml`), `application_settings.yml`, `credentials.yml`, `bot_user.yml`, `groups.yml`, `repos.yml`, `deploy_keys.yml`, `memberships.yml`, `bootstrap.yml`, `runtime_config.yml`, and `push_repos.yml`. The order is load-bearing: the instance must be up and its settings applied before credentials can be minted, and the bootstrap state can only be written once every credential it records exists.

Dockerised GitLab CE. GitLab CE runs as a Docker container from the `gitlab/gitlab-ce:latest` tag (newer versions of the code pin `gitlab/gitlab-ce:19.2.0-ce.0` by digest), with mapped volumes for repositories, registry data, and application config. The play waits for the container’s health probe before proceeding. Because the tag is mutable and the play pulls it on every converge, the running version is whatever `latest` pointed at during the most recent run.

Root token and bot user. At first run the play retrieves the initial root token from GitLab’s disk, creates a dedicated bot user (`handin-bot`), and mints the backend’s service PAT – named `handin-backend-<stack>` after the stack it belongs to – with the scopes the backend needs. Subsequent runs detect the bot user already exists and reuse its record.

PAT and OAuth rotation. If a PAT or the OAuth client is missing, expired, or explicitly marked for rotation, the play recreates it and updates the bootstrap-state file with the new value. Because the CI and web plays render environment files from that state, a single GitLab-play re-run followed by the CI and web plays rotates every credential end-to-end.

System hook and secret. The play registers a system hook pointing at the backend’s `/api/system-hooks/gitlab` endpoint, protected with a shared secret written to the bootstrap state and later consumed by the web play.

Namespace bootstrap. `groups.yml` materialises the `<university>/` root group, the top-level utility project `<university>/provisioner` (which also carries `username-regex.txt`), and the `<university>/ci-templates/` and `<university>/project-templates/` groups. The provisioner project’s initial contents are pushed via `push_repos.yml` from vendored template state, so a fresh GitLab comes up with the provisioner CI already wired.

Container registry, application settings, and limits. The container registry, the external URL, and TLS are part of the omnibus configuration assembled in `gitlab.yml`, because they are container startup concerns rather than API-level settings. `application_settings.yml` then applies the instance settings through the API: the PAT prefix, the OAuth-application and

group-creation locks, signup and e-mail policy, the domain allowlist, and the artefact expiry and size caps. Protected branches on the seeded repositories are set in `push_repos.yml`.

State repo and daemon. A dedicated GitLab repo is created to persist the admin daemon’s `StateStore` snapshot; the daemon consumes it on start.

Handoff. All generated credentials – backend PAT, provisioner trigger token, OAuth client secret, system-hook secret, runner registration tokens – land in the host’s bootstrap-state file under a well-known path. The CI and web plays reference that path through Ansible facts.

7.4 CI Play

The CI play converges `greyfox-ci` into two logical parts: the runner fleet and the Nexus registry cache, with optional stress-test infrastructure alongside.

Runner registration. The play registers two named runners against the GitLab instance using registration tokens taken from the bootstrap state. Both use the Docker executor. Runner tag mapping is explicit: the `provisioner` runner picks up jobs tagged `provisioner` (the provisioner CI project applies this tag to its jobs), while the `ci` runner picks up jobs tagged `ci` (student, teacher, and tutor jobs). This tag-based split is what makes the trust boundary in [section 5.8](#) deploy-time verifiable rather than convention-based.

Trusted versus untrusted volumes. The `provisioner` runner’s Docker-volume list includes `/var/run/docker.sock` so image-build jobs can drive the host Docker daemon; the `ci` runner’s list does not. Ansible defaults make this per-runner so the untrusted default is inherited when no override is provided.

Pull policy. The `ci` runner’s `pull_policy` is `[always, if-not-present]`: Docker checks for a teacher-published image update first, but can fall back to a cached image during a transient registry outage. The provisioner runner uses `[always]` because its jobs are trusted and should fail loudly if the configured base image cannot be refreshed.

Resource caps. `gitlab_runner_docker_cpus_by_runner`, `gitlab_runner_docker_memory_by_runner`, and `gitlab_runner_docker_pids_limit_by_runner` render as per-job CPU shares, memory ceilings, and PID limits. These bound a malicious or accidentally runaway student job.

Tmpfs mounts. The role renders per-runner tmpfs entries for `/tmp` and `/var/tmp`. The configuration still carries `tmpfs_builds` sizing variables, but the current task intentionally does not render `/builds` as tmpfs because GitLab Runner already mounts the build directory and a second mount collides at job startup. Backing temporary directories with RAM keeps

short-lived scratch writes off disk, while the build tree and caches remain on the runner volumes.

Nexus proxies. The `roles/nexus/` play deploys Nexus as a Docker container and provisions three proxy repositories: a Maven proxy (`maven-public` pointing at Maven Central), a PyPI proxy, and a Docker Hub proxy pointing at `registry-1.docker.io`. Student and teacher jobs configure Maven and pip to pull through Nexus; Docker uses the Nexus endpoint only where the runner or workload explicitly configures that registry mirror. This ensures that packages fetched once can be served locally to subsequent jobs.

k3d stress-test infrastructure. Three further roles provide a scratch Kubernetes API for stress-test workflows: `roles/k8s_base/` installs k3d and `kubectl`, and `roles/k8s_prov/` and `roles/k8s_ci/` each stand up a cluster on `greyfox-ci`. They are used only by the optional stress-test path, but the CI play currently converges them unconditionally; gating them behind a group variable remains future work.

7.5 Web Play

The web play converges `greyfox-web`.

Caddy. Caddy terminates TLS, serves the static frontend, and reverse-proxies `/api` to the backend container. It runs as a host service rather than a Compose service, so a stack restart does not take the site down. Its configuration is templated from `roles/web_stack/templates`; the certificate is the university-issued PEM pair described below.

Backend environment file. The backend runs from a container image published to [GitHub Container Registry \(GHCR\)](#); its environment file is rendered by the play from the bootstrap-state values (OAuth client ID/secret, GitLab URL, backend service PAT, provisioner trigger token, invitation secret, admin-daemon URL and bearer secret) plus static group-var values (session TTL, rate limits, trusted proxies). Because the environment file is rendered by Ansible on each web-play run, credential rotations in the GitLab play propagate automatically.

Frontend and backend GHCR images. GitHub Actions builds and publishes container images to GHCR on every push to `main`. The web play pulls the images from GHCR using host credentials rendered by the play. Actual deployment is a manual step ([section 7.7](#)): `/usr/local/bin/deploy-handIn` (rendered from `roles/web_stack/templates/deploy.sh.j2`) pulls the latest images and restarts the Docker Compose stack.

Redis. Redis runs as another Compose service, started with `--appendonly yes` and a persistent volume holding the append-only file, so a restart replays the log rather than losing everything written since the last snapshot. The backend connects over the loopback address; Redis is not exposed on the external interface.

TLS certificate handling. Production uses static PEM material issued by the university [Public Key Infrastructure \(PKI\)](#): the certificate and key are installed on the host and referenced by `web_stack_caddy_tls_certificate` and `web_stack_caddy_tls_key`, and Caddy emits a `tls` directive only when both are set. Leaving them empty yields an HTTP-only site, which is the development default; Caddy’s automatic [Automatic Certificate Management Environment \(RFC 8555\) \(ACME\)](#) issuance is deliberately not used on either production host.

Trusted proxies. The backend’s `trusted_proxies` setting is rendered from the web-play group vars to the address of Caddy on the same host, so `X-Forwarded-For` is honoured only from that one hop. Every other client’s socket IP wins.

Admin-daemon CA file. The backend talks to the admin daemon over TLS with a certificate signed by an internal CA. The web play renders this CA into a file that the backend references through `admin_daemon_ca_file`. This binds trust to the daemon’s specific certificate rather than the system trust store.

Manual deployment model. The web play is idempotent, but the final “pull new images and restart” step is a deliberate manual operation performed with `deploy-handIn`. This separates image publication (which the GitHub Actions workflow does automatically) from production rollout (which stays under human control). See [section 7.7](#) for the trade-off discussion.

7.6 Firewall, TLS, and Network Boundaries

nftables. The `roles/nftables/` play renders a per-host `nftables` ruleset with an explicit default-drop policy on the input chain and rules that permit only the ports actually served by that host’s role. The rules are rendered from Ansible facts (which service is enabled on this host?) so a stale rule does not survive a role removal.

GitLab SSH port and host admin SSH port. GitLab in a Docker container needs host TCP 22 to bind for git-over-SSH, so on `greyfox-git` the host’s own `sshd` is moved to port 2222 (the `common` role does this wherever `host_admin_ssh_manage` is enabled, which in the production inventory is that host alone). The arrangement is not free: the university firewall permits inbound SSH only on the canonical port from outside the campus network, so administrative access to `greyfox-git` on 2222 requires being on the university network, and giving GitLab the canonical port is what forces that trade-off ([section 10.2](#)).

GitLab registry port. The GitLab Container Registry is exposed on TCP 5050. The `nftables` ruleset opens this port on `greyfox-git` only.

Admin-daemon source-IP allowlist. The admin daemon binds on a private interface and is only reachable from the backend host. `nftables` enforces this with an explicit source-address

match; the daemon also performs its own bearer-secret check on every request, so the network allowlist is defence-in-depth rather than the only barrier.

Caddy TLS with university-issued PEMs. Both externally reachable hosts – `greyfox-web` (frontend and API) and `greyfox-git` (GitLab web UI and container registry) – run Caddy with a per-host PEM pair issued by the university PKI; the private key never leaves the host it was issued for. Caddy’s ACME path is disabled in this mode to avoid accidental re-issuance during teaching hours.

Backend trusted proxies. As covered in [section 5.5](#), the backend trusts `X-Forwarded-For` only from configured proxies. In production that set is exactly the local Caddy address; in development it may include a docker-compose bridge address. Any header from an unrecognised peer is ignored and the socket IP wins.

7.7 Deployment Pipeline

GitHub Actions builds the backend and frontend container images from the source repository and publishes them to GHCR. Production deployment is intentionally manual: an operator runs the Ansible/web deployment or the rendered deploy script so that code publication and production rollout remain separate steps.

The deliberate split is framed here as a safety trade-off. Automatic production rollout on every push to `main` would remove the operator step but also remove the natural rollback window between “image published” and “image serving students.” In a course setting, a broken deploy that catches five minutes of student traffic is far worse than a five-minute delay before rollout. The image build in GitHub Actions is fully automatic; only the last step is intentionally human-gated. That said, the current shape – an operator SSHes into `greyfox-web` and runs `/usr/local/bin/deploy-handIn` – would benefit from a gated but push-button surface (for example, a signed-approval workflow that fires the same script). This is captured in [section 10.3](#).

Chapter 8

CI-Based Assessment and Reproducibility

This chapter covers what happens when a student presses *compile* or *test*, and how the result is made both reproducible and safe to show. [Section 8.1](#) describes the per-language project and CI templates from which student repositories are derived, and [section 8.2](#) the compile and test jobs those templates provide. [Section 8.3](#) separates these student-visible jobs from the private grader and its package registry flow, which must remain unreadable from a student pipeline. [Section 8.4](#) explains the per-assignment container images that pin a fixed toolchain and pre-baked test artefacts to an assignment version, and [section 8.5](#) discusses the caching and performance behaviour that makes this practical when many short jobs run in sequence.

8.1 Language Templates

The platform contains separate project templates per language (Java/Maven and Python are the implemented paths). The CI templates and graders, by contrast, are consolidated into a single `<university>/ci-templates` repository: the per-language compile/test jobs live in `/java-maven` and `/python` subdirectories, and the grader is that repository's root `.gitlab-ci.yml`, routed by a `language` input. Child/student templates strip teacher-only solution material and include the appropriate per-language file from the merged repository.

8.2 Student Compile and Test Jobs

The workspace exposes three explicit CI actions: *compile*, *test*, and *publish tests to registry*. Each is a distinct button that maps to a distinct provisioner-safe pipeline input, so ambient saves cannot accidentally trigger a graded run.

HANDIN_ACTION. The student CI templates (the per-language files `/java-maven/*` and `/python/*` in the single merged `<university>/ci-templates` repository) declare a top-

level `spec:inputs` block with an `action` field. The pipeline stores that input in a plain CI variable `HANDIN_ACTION`, and each job's `rules` guard on that variable. This gives the platform a small, enumerable set of graded verbs while still letting GitLab route them through its native pipeline-input machinery.

Branch and ref handling. Compile and test run against the current WIP branch; publish always runs against `main` on the mother project. The backend enforces this: the mother's publish endpoint rejects any `ref` other than the configured default.

Compile/test separation. Compile and test are separate jobs in separate CI stages, sharing the runtime image but not sharing state. The rationale is diagnostic: a student who breaks compilation should see that failure without ambiguity, and a student whose code compiles but fails tests should see the test log without wading through compile output. Each job's log is streamed to the workspace with a severity-filtered highlight pass so meaningful lines (errors, failures, passed/failed counts) stand out from the runner boilerplate.

Artefacts. Test jobs emit JUnit XML (Java) or the pytest JSON report (Python) as GitLab artefacts. The backend reads these from the pipeline artefact API and turns them into typed `TestSummary` and `TestCaseResult` objects for the frontend to render as a table.

Skip-CI on normal saves. A file save is not a graded action: the student is snapshotting work. The save commits carry a `[skip ci]` tag in the commit message so GitLab does not spawn a pipeline for every keystroke-save. Explicit compile/test actions are the only way to launch a graded pipeline, which keeps the CI cost predictable and keeps the shared runners free for jobs the student actually asked for.

8.3 Private Grader and Package Registry Flow

Earlier iterations placed a grader trigger token in a variable readable by student CI jobs so that student pipelines could hand off to a grading pipeline. That approach was retired because it let any leaked student log or logged variable expose the token, and rotation required touching every student project.

Private grader project. The current design keeps the grader outside the student's reach: it is the root `.gitlab-ci.yml` of the merged `<university>/ci-templates` repository (formerly a separate per-language `grader` project), invoked through a pipeline trigger using a single trigger token and a `language` input that routes to the matching runner. The student's compile and test jobs do not run the graded logic themselves; they invoke the grader's CI with a small, typed input set: `action`, `source_project_id`, `source_ref`, `source_sha`, `source_pipeline_id`, `source_job_id`, and `source_token_valid_until`. GitLab's `spec:inputs` declaration constrains `action` at pipeline start; the grader-side `scripts/validate_request.py` re-validates every input at run time as defence-in-depth.

Job-token allowlist as the trust primitive. Instead of a long-lived shared token, the student pipeline authenticates with the grader using its own short-lived `CI_JOB_TOKEN`. The grader project’s job-token allowlist accepts calls only from source projects under the configured course groups. This turns “who may hand off to the grader” into a GitLab-native, per-project allowlist entry that the provisioner adds when a new mother is created and removes when a course ends.

Package registry / Maven test JAR flow. For Java/Maven assignments, the mother project’s publish pipeline builds the compile-time test JAR and pushes it to the mother’s own GitLab Maven Package Registry. The student pipeline resolves that JAR through Nexus during the compile job, so the student never needs Maven credentials, only Nexus access. The published JAR is content-addressed by the mother project and version.

Python equivalent. For Python assignments the same idea uses a PyPI-format wheel pushed to Nexus’s hosted PyPI repository. The student’s compile job resolves the wheel with a normal `pip install`, again via Nexus.

Baked-into-image path. When the assignment image ([section 8.4](#)) is baked with the freshly built test artefact already installed, the compile-time resolution collapses to a no-op: Maven finds the JAR in the local `.m2`, pip finds the wheel already installed. This makes the runtime install path a safety net rather than the hot path.

8.4 Per-Assignment Docker Images

Per-assignment images improve environment-level reproducibility by pinning the operating system, toolchain, and grader runtime for an assignment. The provisioner records the initially built image reference and digest in `.handin/assignment.toml`; child-template creation turns that into `HANDIN_ASSIGNMENT_IMAGE`, digest-pinned when a digest is available. The teacher publish pipeline later bakes the freshly published test artefact into the assignment image tag via `kaniko`, but the current implementation validates that tag rather than writing the post-publish digest back into `.handin/assignment.toml` or the child group variables.

Per-assignment images solve three problems and explicitly do not solve three others. [Figure 8.1](#) traces the flow from teacher publish to student compile job.

Solved. First, they pin the operating system and toolchain versions (the JDK, Maven distribution, or Python interpreter, plus their transitive system packages), so a student on one machine and a grader six months later see the same runtime. Second, at child-template creation time they can pin the grader’s baseline libraries to a digest-pinned image value, so the configured runtime environment cannot drift under the grader’s feet. Third, when the configured image already contains the relevant test artefact, they eliminate a network round-trip on every compile job during peak load.

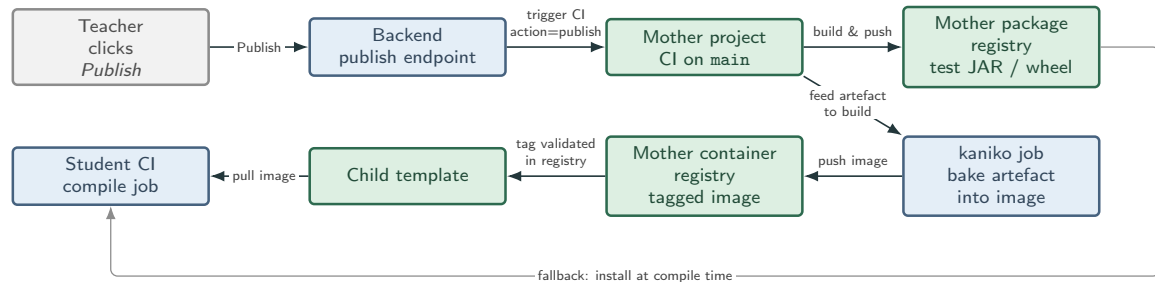


Figure 8.1: Per-assignment image build and propagation. The mother project’s publish pipeline produces both the language-native artefact (Maven JAR / Python wheel) and the assignment image containing that artefact. Child templates reference the image value recorded at child-template creation time, digest-pinned when the initial provisioner build recorded a digest. The publish pipeline validates the pushed tag and baked artefact, but does not yet propagate the new digest back to the child variables. The muted arrow shows the runtime fallback path when the configured image does not contain the latest test artefact.

Not solved. Dependency *lockfiles* inside the student repository are a separate concern: without one, Maven or pip is free to resolve to a newer transitive dependency the moment upstream publishes one. The image pins the resolver, not what it resolves. Full *offline* reproducibility – reproducing a graded run six months later without external network at all – would additionally require the image to contain the full transitive dependency closure, which the current implementation does not enforce. Finally, the current publish path pushes and validates the assignment image tag, but it does not yet back-propagate the new digest into `.handin/assignment.toml` or the child group variables. These limitations are noted in [section 10.3](#).

8.5 Caching and Performance Considerations

Performance in this platform is dominated by three quantities: how much each job downloads over the network, how much it writes to disk, and how often it re-pulls its base image. Each has a targeted mechanism.

Nexus. Maven and pip dependency fetches are routed through Nexus proxies on `greyfox-ci` (Maven Central and PyPI; Docker Hub only where explicitly configured; see [section 2.5](#)). First-hit downloads pay the upstream cost once; subsequent jobs are served locally. This turns the failure mode from “every student’s compile job hits Maven Central in the last minutes before the deadline” into a bounded local hit.

Maven cache keys. Each student CI pipeline caches the local `.m2/repository` keyed by `pom.xml`. When the POM is unchanged the local cache short-circuits Nexus entirely. Cache archives are keyed per project, so no student’s pipeline *resolves* another’s cache under normal operation. That scoping is a naming convention rather than an isolation boundary.

Runner image pull policy. The ci runner’s pull policy is `[always, if-not-present]`, so a teacher-published image update is checked on the next job while a cached image can still run through a transient registry outage. In practice Docker’s manifest-check-then-pull means only new digests are actually downloaded, so the leading “always” check is cheap in the common case.

Tmpfs scratch directories. `/tmp` and `/var/tmp` are RAM-backed (see [section 7.4](#)); the build directory and runner caches remain on runner volumes. This keeps short-lived scratch writes off disk and makes their cleanup instantaneous when the container disappears, without claiming that the checkout tree itself is tmpfs.

Deferred: shared runner cache. A tempting further step is a shared cache directory across every student’s pipeline for the same assignment: one download of a large dependency serves everyone. This is deliberately deferred because a shared cache is a poisoning surface. A malicious student could inject a modified JAR into the cache path, and the next student’s job would resolve to the modified artefact. Solving this cleanly requires per-artefact integrity checks and a write-once cache policy that goes beyond the default runner cache behaviour, so it is treated as future work.

Chapter 9

Evaluation

This chapter asks how far the implemented system meets the requirements of [chapter 3](#) and holds against the threat model of [section 5.1](#). [Section 9.1](#) states the strategy and its deliberate limits, after which [sections 9.2](#) and [9.3](#) report the automated test layers and the integration playbook exercised against a running stack. [Section 9.4](#) revisits each threat and records where it is countered and by what mechanism. [Sections 9.5](#) and [9.6](#) turn to the deployed system itself – operational validation and estimates of storage, idle resource use, and per-scenario resource use – and [section 9.7](#) states what this evaluation does not establish.

9.1 Evaluation Strategy

The evaluation combines code-level tests, integration tests, an architecture review against the threat model, and operational validation on the deployed stack. It makes no claim to a formal security proof: what follows states what is tested, what is inspected by review, and what remains a residual risk.

9.2 Automated Test Coverage

The project contains four test layers. [Table A.4](#) summarises their size and measured coverage as of thesis submission; the per-layer detail and frameworks are collected in [table A.5](#). The three code-level suites (backend, provisioner, frontend) are reported as line coverage; the Ansible integration layer drives a deployed stack through its real interfaces rather than importing code, so it carries no line-coverage figure and is instead reported by the breadth of workflows and endpoints it exercises ([section 9.3](#)). Counts are reproducible – the test-case totals via `pytest--collect-only` (Python) and the Vitest run summary (frontend), and coverage via the commands noted in [table A.4](#) in the appendix.

At the time of thesis submission, the suites can be reproduced as follows. Backend and provisioner tests run with `pytest -q` from their respective virtual environments; frontend tests run with `npm test` (or `npm run test:coverage`) inside `frontend/`; the Ansible in-

tegration test runs against a live `<university>_prod_*` stack with `ansible-playbook -i inventory/inventory.yaml playbooks/integration_test.yaml`. CI enforces the coverage floor of 80% on the backend suite (`.github/workflows/ci.yaml`); dropping below that fails the workflow, and the measured 95% leaves a comfortable margin above it. Live-run pass/fail status is captured in [section 9.5](#).

9.3 Integration Test Playbook

The Ansible integration-test playbook exercises a deployed stack against an isolated namespace.

How integration coverage is expressed. Unlike the unit suites, this layer drives the system through its real [HTTPS](#) and GitLab interfaces rather than importing modules, so a line-coverage percentage is neither collected nor meaningful for it. Integration coverage is therefore stated as the *breadth of user-facing workflows and backend endpoints the playbook drives to completion against a live stack*: a task “covers” a flow when it performs the real request sequence and asserts on the resulting GitLab state, membership, pipeline outcome, or HTTP status. The 48 task files span the following surface.

- **Platform bootstrap:** health and readiness endpoints, registration and tags of the `ci` and `provisioner` runners, and administrator creation of a department and course.
- **Assignment lifecycle:** mother creation for both language toolchains (Java/Maven and Python), child-template creation, assignment configuration, publishing of the compile-test artefact, per-assignment image build, container-registry tag lookup, baked-image validation, and template and student-code update propagation.
- **Invitation and access:** single-student `CLAIM_CREATE`; the group flow (`CLAIM_CREATE` + `CLAIM_JOIN` with a full-group rejection); post-deadline rejoin; student repository access; teacher and tutor staff management; and proctor and test-mode workflows.
- **CI and grading:** student compile/test pipelines, deadline grader dispatch, and the optional stress-test path.
- **Security and negative paths:** CSRF rejection, `Idempotency-Key` replay, wrong-trigger-token rejection, cross-student repository isolation, outsider read denial, the anonymous-access sweep asserting that every protected backend endpoint rejects unauthenticated requests, IP-allowlist enforcement, the signup/username policy, and the SMTP notification filter.
- **Authentication surface:** the OAuth login redirect (and the GitLab-host config-drift check it implies), the CLI device-flow `start/poll` steps, logout, and workspace browsing.
- **Browser-driven UI (opt-in):** a Playwright suite that drives the *rendered* SPA in a headless GitLab-authenticated Firefox – the OAuth login and logout through the real sign-in form, the workspace editor with the Compile/Test/Save actions and the

role-gated Publish button, the invitation claim page end to end, and the teacher and administrator provisioning views. Each scenario asserts on the rendered DOM *and* the API calls the buttons fire, and captures a screenshot plus a Playwright trace on failure.

The browser suite drives the authenticated *in-browser* OAuth completion directly: the claim scenario, for instance, logs a student in through the GitLab sign-in form and completes a repository claim entirely in the browser. It runs headless Firefox rather than Chromium, which aborts the [greyfox-web](#) redirect chain with `ERR_ABORTED` on this network, and is opt-in (`-e integration_test_run_ui=true`) because the Firefox bundle is a heavier dependency and DOM assertions are more sensitive to interface refactors than pure API checks; stable `data-testid` attributes on the driven controls keep the locators resilient. Finally, the whole playbook is kept structurally sound between live runs with `ansible-playbook [...] --syntax-check`, which parses every task file without contacting a host.

Because the playbook drives the *live* backend – which is scoped to a single `PROVISION_ROOT_GROUP` (`<university>`) – the suite cannot run in a genuinely separate “integration-test university”; a parallel root group would be invisible to the running backend. Isolation is instead achieved by confining every test artefact to a reserved `e2e-<epoch>` namespace under `<university>` and deleting it afterwards. One consequence is that the suite’s request traffic shares the production backend’s log stream. Since all services log to `stdout` and are bounded only by Docker’s `json-file` rotation, a burst of test traffic would otherwise rotate genuine operational log lines out of the retained window. To prevent this, the backend installs a `NamespaceExclusionFilter` (`backend/app/core/logging.py`) that drops log records referencing the reserved namespace; production enables it with `LOG_EXCLUDE_NAMESPACE=e2e-`. Setting the variable empty restores full logging when a suite failure must be debugged.

9.4 Security Evaluation Matrix

The threat/defence matrix in [section 5.10](#) is augmented here with the concrete implementation and test evidence for each row.

Threat	Defence	Implementation evidence	Test evidence
Compromised backend	Service PAT is read-oriented; admin ops go through the daemon with bearer secret over TLS.	<code>backend/app/services/admin_daemon.py</code> , <code>admin-daemon/handin_admin_daemon/auth.py</code>	<code>ansible/roles/integration_test/tasks/admin_daemon.yml</code> (network gate + bearer)

Threat	Defence	Implementation evidence	Test evidence
Malicious student CI	ci runner without host Docker socket; tmpfs; resource caps.	ansible/roles/gitlab_runners/defaults/main.yml	ansible/roles/integration_test/tasks/repo_access.yml, runner registration inspection
Forged invitation	HMAC-signed payload, constant-time verify.	backend/app/services/invitations.py	backend/app/tests/test_invitations*.py
Cross-student access	Namespaces per student; membership granted only to claimant.	provisioner/scripts/invitation_dispatch.py	ansible/roles/integration_test/tasks/negative.yml (outsider, cross-student)
Spoofed X-Forwarded-For	IP resolver honours forwarded headers only from trusted proxies; test-mode uses socket IP.	backend/app/request_control/identity.py	backend/app/tests/test_identity*.py, ansible/roles/integration_test/tasks/ip_allowlists.yml
Leaked trigger token	Token starts one project's pipeline only; provisioner validates every input.	provisioner/scripts/validate_inputs.py	provisioner/tests/test_validate_inputs.py
Stale membership	GitLab expires_at ends write access; daemon schedules the post-deadline run and exact-instant staff revocation.	admin-daemon/handin_admin_daemon/expiry.py	ansible/roles/integration_test/tasks/deadline_tests.yml, deadline_update.yml
Invalid user-name signup	GitLab sends user_create with the configured system-hook token; backend and daemon validate against regex.	backend/app/services/username_policy.py, admin-daemon/handin_admin_daemon/policy.py	ansible/roles/integration_test/tasks/signup_policy.yml

9.5 Operational Evaluation

The following operational checks were run against the production stack on 2026-07-13 (last full regression 2026-06-23) as the final acceptance test. Each check is reproducible from the repository without hidden manual setup. The software under test was [HandIn](#) 0.9.0-beta.

- **GitLab CE version.** GitLab Community Edition, deployed from the configured image `gitlab/gitlab-ce:latest`; the Ansible GitLab role pulled that tag on convergence rather than pinning it to a recorded digest (newer versions of the code pin `gitlab/gitlab-ce:19.2.0-ce.0`).
- **Runner presence.** Two runners register against the instance and appear online: the `provisioner` runner with Docker socket, and the `ci` runner without. Verified via the `/admin/runners` view and the integration test's `runner-tag` probe.
- **Java/Maven assignment path.** A mother is created, tests are published to the mother's Maven registry, the assignment image is built into the mother's container registry, an invitation is claimed, a student pipeline compiles and tests, and the result artefact is surfaced in the workspace.
- **Python assignment path.** Same flow with a wheel in the hosted PyPI proxy instead of a Maven JAR.
- **Registry and image validation.** The container registry serves the expected assignment image tag, the runner host can pull it, and the baked provenance/test artefact is present in the image.
- **OAuth login redirect.** A fresh browser hitting the base URL is redirected through the GitLab authorisation endpoint and returned with a session cookie; the CSRF cookie is set.
- **Frontend and backend health.** `/api/health` returns 200 and the frontend static bundle loads without console errors.
- **Cleanup.** The integration-test playbook creates an isolated `e2e-<epoch>` namespace and, on success, removes every GitLab object it created. A subsequent list of `<university>/` groups shows no leftover `e2e-*` entries.
- **Log retention bound.** Every container's logs are capped by Docker `json-file` rotation at `max-size 512m × max-file 6` (≈ 3 GiB per container), configured in the `docker` and `gitlab` Ansible roles. Combined with the namespace log filter above, an integration-test run leaves the retained production log window intact.

9.6 Capacity and Resource Usage

This section sizes the platform for a realistic teaching load. Two kinds of figure appear, and they are kept clearly apart. The persistent *storage* estimates in [section 9.6.1](#) are the course

staff’s own planning numbers for two representative courses; the *compute and traffic* figures in [sections 9.6.2](#) and [9.6.3](#) are engineering estimates derived from the configured container resource caps ([section 7.4](#)), GitLab’s published sizing guidance, and the image sizes noted in [section 8.4](#). They are labelled as estimates throughout and were not obtained from long-run production telemetry; establishing measured baselines is noted as future work in [section 10.3](#).

9.6.1 Persistent Storage for a Standard Course

Two courses bracket the expected range: a small project course (*Software Engineering*, few large team repositories) and a large introductory course (*Introduction to Programming*, many small individual repositories). [Table 9.2](#) gives the per-semester estimates.

Course	Groups	Teams/grp	Assign.	Student-repo payload	Assignment images
Intro to Programming	5	30	15	≈ 250 MB per group $\Rightarrow \approx 1.25$ GB total	$\approx 15 \times 0.5$ GiB ≈ 7.5 GiB
Software Engineering	2	15	10	≈ 100 – 150 MB per group $\Rightarrow \approx 250$ MB total	$\approx 10 \times 0.5$ GiB ≈ 5 GiB

Table 9.2: Per-semester storage estimate for two representative courses. Student-repo payloads are course-staff planning figures (starter and test code ≈ 200 KB each per assignment; small individual repos ≈ 0.5 MB). Assignment-image figures are estimates: mother container-registry images are ≈ 500 MiB each and are created roughly once per assignment version, shared across all groups and students of the course.

The Git payload of student work is modest – at most a few gigabytes per course per semester even for the large course. The dominant per-course storage term is instead the per-assignment container images in the mother’s registry ([section 8.4](#)); these are bounded by the number of assignment *versions*, not by enrolment, because every student and team of an assignment pulls the same image. Installation-wide overhead is fixed rather than per-course and is not attributed to either course above: GitLab’s bundled PostgreSQL, CI job artefacts, and rotated logs bounded at ≈ 3 GiB per container ([section 9.5](#)).

9.6.2 Idle Baseline Over Time

At rest – no student jobs, no provisioning – the platform’s resource use is flat and dominated by GitLab CE. The following are steady-state estimates, not measurements:

- **RAM.** GitLab CE (Puma, Sidekiq, Gitaly, PostgreSQL, its bundled Redis and nginx) is the resident-set floor on **greyfox-git**, on the order of a few GiB for a tuned single-instance deployment; the backend, frontend, Caddy, admin daemon, and the application Redis add a few hundred MiB combined. On **greyfox-ci**, idle runner resource use is near zero, while Nexus holds roughly 0.5–1 GiB resident.

- **CPU.** Near-zero at idle apart from GitLab’s periodic Sidekiq cron jobs, which produce short, low sub-core spikes rather than sustained load.
- **Network.** Negligible: background health checks and the occasional scheduled job. External (internet) egress at idle is effectively zero.
- **Disk over time.** The only quantity that grows while idle is on-disk history (CI artefacts, container logs). Growth is bounded by the artefact-expiry policy and the Docker `json-file` rotation caps described in [section 9.5](#), so idle disk use is monotonic but capped, not unbounded.

9.6.3 Resource Use by Run Scenario

Active load arrives in bursts tied to the assignment lifecycle rather than as steady traffic. [Table A.6](#) in the appendix estimates the dominant resource for each characteristic scenario. “Internet traffic” means egress to and ingress from outside the three-host network; most inter-host traffic stays on the LAN, and upstream package and base-image fetches are served from the on-site Nexus proxy where the relevant package manager or runner mirror is configured for it, so their external cost amortises to near-zero on the hot paths.

The practical consequence is that the platform is provisioned for *bursts*, not for a high steady-state load: peak demand coincides with an assignment opening (claim burst) or closing (grader batch), both of which are bounded – the claim burst by GitLab’s membership-write throughput and the backend’s request-control limits, and the grader batch by the runner’s job concurrency. Between these events the system returns to the flat idle baseline of [section 9.6.2](#).

9.7 Limitations of the Evaluation

- Unit tests do not prove the full deployed multi-host configuration.
- The opt-in Playwright suite drives the main authenticated browser paths (login, workspace, claim, provisioning views), but does not yet cover every OAuth error branch or run across multiple browser engines.
- GitLab CE behaviour and API details can change across versions.
- The integration suite runs against the production backend rather than an isolated one, because that backend is bound to a single root group ([section 10.2](#)). Confinement to a reserved `e2e-` namespace is a convention the suite observes, not a boundary the system enforces, so a defective test can in principle touch state outside it.
- Some operational checks require live host access and cannot be reproduced purely from the repository.

Chapter 10

Discussion

This chapter steps back from the implementation and reflects on the choices behind it. [Section 10.1](#) revisits the central design decisions – among them the absence of a separate application database and the delegation of privileged writes – and states what each one costs as well as what it buys. [Section 10.2](#) collects the limitations that remain in the delivered system, as opposed to the limitations of its evaluation discussed in [section 9.7](#), and [section 10.3](#) outlines the work that would address them.

10.1 Design Trade-Offs

- **No separate application database.** This improves inspectability and reduces state synchronisation problems, but pushes more modelling work into GitLab groups, files, variables, and daemon state.
- **Provisioner CI as privileged executor.** This gives logs, artefacts, retries, and credential separation, but adds queueing latency and splits features across backend routes and CI scripts.
- **Manual production deployment.** This avoids accidental automatic rollout, but leaves deployment dependent on operator action.
- **Per-assignment images.** This improves runtime reproducibility, but full dependency reproducibility still needs lockfiles or baked dependency closure.
- **GitLab Community Edition.** CE keeps the stack self-hosted and accessible, but some features such as protected container tags or advanced policy management are limited.

10.2 Known Limitations

The following limitations are current at submission. Each remains unresolved because addressing it would have exceeded the thesis scope.

- Deadline updates must be carefully checked against existing student memberships.
- Stress-test workflows are less central than assignment provisioning and may not have the same production maturity.
- Shared dependency cache across all student repositories is deferred because cache poisoning must be avoided.
- Some operational tasks such as certificate renewal and host SSH migration remain operator-driven.
- Implementing a privileged feature requires coordinated changes to both the backend and the provisioner.
- Assignment images are large; a first pull on a cold runner is slow, and warm-pull assumptions rely on the runner host having a persistent Docker image cache.
- Production deployment is manual (`/usr/local/bin/deploy-handin`); no signed-approval CI path exists yet.
- The platform has been exercised against Java/Maven and Python; additional language toolchains would each require another project template and a new per-language route in the shared grader pipeline.
- A backend instance serves exactly one institution. `PROVISION_ROOT_GROUP` is a single group path, and role resolution, course authorisation, and namespace construction all key off it, so a second university means a second deployment rather than a second root group inside the same one. This is adequate for the department the platform was built for, but it also means the integration suite cannot isolate itself in a parallel root group ([section 9.3](#)).

10.3 Future Work

- Optional admin service that handles provisioner-CI operations directly when it is reachable, with the provisioner pipeline retained as a fallback – avoiding the two-project split and the user-visible pipeline latency on claim and template creation in the common case, without giving up the current behaviour when the service is unavailable.
- Digest write-back after the publish-time kaniko bake, completing image provenance down to exact bytes rather than only to the tag. The bake currently replaces the image behind `HANDIN_ASSIGNMENT_IMAGE` without updating `HANDIN_ASSIGNMENT_IMAGE_DIGEST` or the `[assignment_image]` section that records it, so a graded run is traceable to a tag but not to the image bytes it actually ran on.
- Fast compile-and-test service – potentially browser-side – primarily aimed at [test mode](#) and exam scenarios where a quick pass/fail turnaround on the student’s own tests is

imperative, decoupled from the CI-based grading path so that time-critical runs do not queue behind graded jobs elsewhere on the same runner.

- Multi-institution operation from a single deployment: making the root group a per-request property resolved from the authenticated user rather than one instance-wide setting, so that several universities – or several faculties administering their own namespaces – can share one installation instead of each running their own three hosts. The same change would let the integration suite provision a genuinely separate test root group instead of a reserved namespace inside the production one.
- Reproducible disaster-recovery rebuild driven from Ansible, including baseline GitLab state restore and integration-test replay against a freshly provisioned three-host set.
- Ticket system linking teacher/tutor and student questions, extension requests, and edge cases to a specific assignment or repository, so that deferrals are tracked next to the artefacts they affect.
- Admin dashboard surfacing system load, runner queue depth, disk headroom, and per-host health, closing the observability gap that currently forces operators to check the runner and web hosts by hand.
- Group-finding feature to help students form teams for group assignments without teacher-side spreadsheets or manual matchmaking.
- Grade export in standardised formats (CSV, LMS-compatible) and richer teacher analytics – per-assignment pass distributions, common failure patterns, and time-to-first-passing-commit histograms – so that teachers can adjust the difficulty of an assignment without piecing the picture together from raw pipeline logs.
- Stronger, fully reproducible dependency snapshots: lockfile-based resolution baked into the assignment image at publish time, so that Maven Central or PyPI turnover between publish and grade cannot silently shift a transitive dependency underneath a student submission.
- Extending the browser-driven end-to-end suite with cross-browser coverage and visual-regression snapshots, and running it in CI against an ephemeral stack so browser-layer regressions are caught before deployment rather than during teaching.
- Additional language templates beyond Java/Maven and Python – C/C++ (CMake), Rust (cargo), and JavaScript/TypeScript (Node) are the most common curriculum requests – each of which requires its own compile-test job definition and per-language runtime image.
- A formal security hardening review and penetration testing pass, ideally by an external party, focused on the OAuth boundary, the admin daemon’s bearer channel, the invitation and rejoin token surface, and the runner isolation configuration.

- Automated but gated production deployment: the existing manual deploy script wrapped in an approval workflow (GitHub Environments or a GitLab manual job) so that a merged **main** is rolled out on a single reviewer approval rather than through a shell session on the web host.

Chapter 11

Conclusion

Programming courses at scale need a workflow that automates repository creation, membership expiry, and CI-based assessment without granting a public web application broad authority over the underlying GitLab instance. [HandIn](#) addresses this by placing a FastAPI backend at the single enforcement point, delegating privileged mutations to a provisioner CI project and an admin daemon, and modelling academic state directly in GitLab objects rather than in a secondary database.

The thesis develops two focus contributions in depth. The first, covered in [chapter 5](#), is the security architecture: no GitLab admin token reaches the browser or the public backend, session and CSRF handling are centralised in Redis, invitation tokens are [HMAC](#)-signed with server-verified replay markers, and the runner trust boundary is enforced at the Ansible-defined executor level rather than by convention. The second, covered in [chapters 6 and 7](#), is the automated lifecycle: an enumerable provisioner action surface, a strict Ansible playbook order that converges rotated credentials from GitLab through CI to the web tier, and per-assignment Docker images baked in the mother project's own container registry.

Evidence for these contributions is threefold. Unit tests exercise the backend, frontend, and provisioner in isolation, while the Ansible integration-test playbook drives the deployed multi-host stack end to end and catches deployment drift that unit tests cannot. The Ansible integration-test playbook drives the deployed multi-host stack end to end and catches deployment drift that unit tests cannot. The production-validation checks in [section 9.5](#) confirm the Java/Maven and Python paths on the live stack.

The most important remaining limitation is the backend–provisioner coupling that follows from the delegated-executor design: any privileged feature ships as two changes, one in each project, and the pipeline latency between them is user-visible. The proposed future work ([section 10.3](#)) would short-circuit that latency by adding an optional admin service that handles the same operations directly when reachable – with the provisioner pipeline retained as a fallback – and adds gated but push-button production deployment; the security and lifecycle principles developed in this thesis would remain unchanged.

Appendix A

Additional Diagrams and Reference Tables

This appendix collects supplementary reference material that does not fit naturally inside the main narrative. The core figures and tables – the three-host overview ([figure 4.1](#)), the per-host internal diagrams ([figures 4.2 to 4.4](#)), the invitation claim sequence ([figure 6.1](#)), the per-assignment image flow ([figure 8.1](#)), the credential placement table ([table 5.1](#)), and the security matrix ([section 9.4](#)) – are placed inline with the prose that refers to them. The namespace structure trees ([figures A.1 to A.3](#)), referenced from the namespace-design discussion in [section 6.1](#), are collected below, followed by five reference tables covering stakeholder roles and access levels ([table A.1](#)), the provisioner action surface ([table A.2](#)), the Ansible roles and host targets ([table A.3](#)), and the test-layer breakdown ([table A.4](#)) and measured coverage ([table A.5](#)).

A.1 Namespace Structure Trees

The three figures below expand the GitLab namespace layout discussed in [section 6.1](#), one per level. Green nodes are concrete projects (Git repositories); blue nodes are groups; dashed nodes mark a subtree that is expanded in the next figure. The trees are conceptual: groups shown once (`<department>`, `<course>`, `<semester>`, `<assignment>`) repeat many times in a live installation.

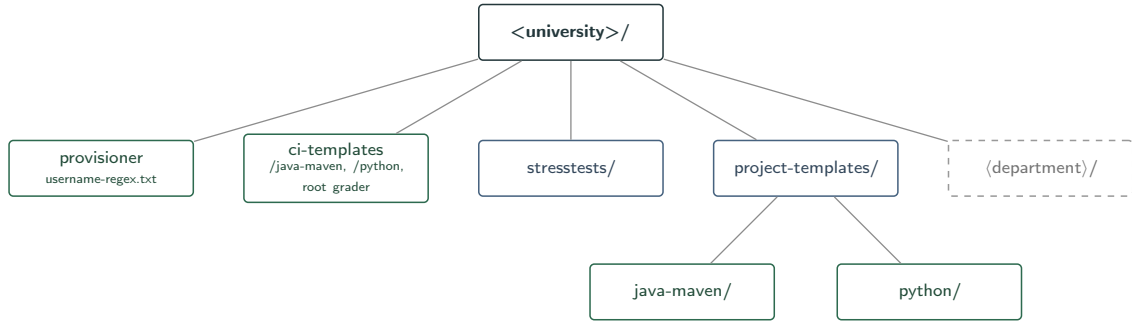


Figure A.1: Platform level: the course-independent projects directly under the `<university>` root group. The `<department>` subtree (dashed) is expanded in [figure A.2](#).

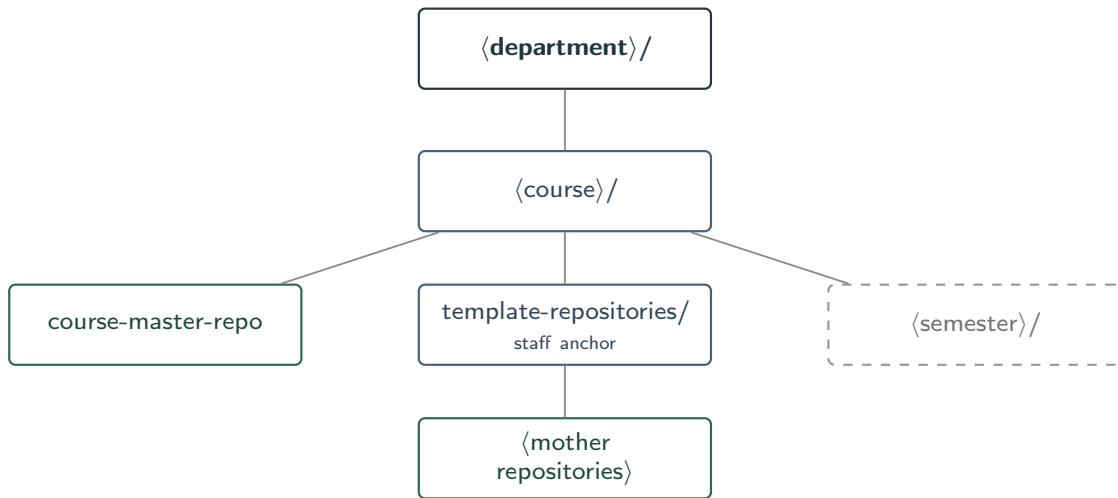


Figure A.2: Course level: a course at `<department>/<course>`, its `course-master-repo`, the `template-repositories` staff anchor holding the mother templates, and its per-semester subgroups. The `<semester>` subtree (dashed) is expanded in [figure A.3](#).

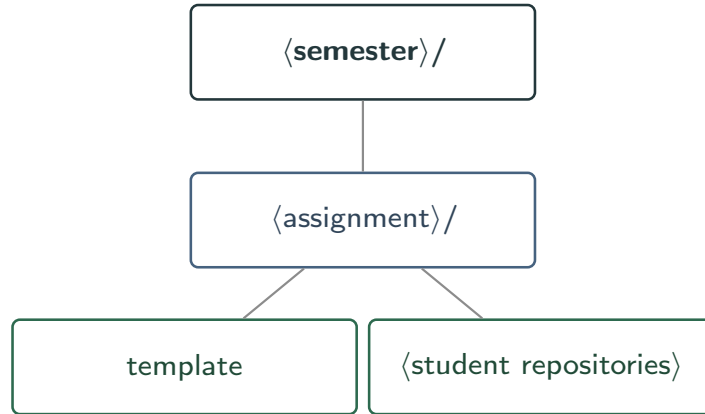


Figure A.3: Assignment level: within a semester, each assignment subgroup holds the child `template` (forked from a mother) and the per-student or per-team repositories created from it.

A.2 Stakeholder Roles and Access-Level Mapping

Role	GitLab anchor	Backend capability
Student	Developer on their student repository, expires at deadline	Claim invitation, save, compile, test, re-join
Tutor	Maintainer on <code><course>/template-repositories</code>	Read student progress, run test/compile on any repo in the course
Teacher	Owner on <code><course>/template-repositories</code>	Create mothers/child templates, invite, publish, manage tutors
Administrator	Configured <code>ADMIN_USERNAMES</code> / <code>ADMIN_USER_IDS</code> , or GitLab admin	Bootstrap departments and courses, add teachers, operational actions
Operator	Host SSH access on <code>greyfox-*</code>	Run Ansible; no in-band platform authority

Table A.1: Stakeholder roles, their GitLab anchor, and the backend capabilities that follow.

A.3 Provisioner Actions and Required Roles

Table A.2: Provisioner action surface, keyed to the role that legitimately triggers each action.

Action	Caller role	Effect
<code>create_department</code>	Admin	Create department group
<code>create_course</code>	Admin	Create course group + <code>template-repositories</code> anchor
<code>create_mother</code>	Teacher	Create mother project from language template; seed <code>.handin/</code>
<code>create_child_template</code>	Teacher	Create assignment subgroup + child template (also used in update mode)
<code>create_student_repo</code>	Operator (CLI only)	Create per-student repository from child template; predates the claim flow, which uses <code>CLAIM_CREATE</code>
<code>PREPARE</code>	Teacher (invitation flow)	Preflight check that the student template project exists before invitations go out
<code>CLAIM_CREATE</code>	Student (via backend)	Create or reuse student repository; add membership; write marker
<code>CLAIM_JOIN</code>	Student (via backend)	Join existing group assignment repository
<code>CLAIM_REJOIN</code>	Student deadline	Restore Guest access via signed rejoin token
<code>build_assignment_image</code>	Bootstrap / operator	Build and push per-assignment image; the teacher's Publish builds it in the mother's own CI instead
<code>run_deadline_tests</code>	Provisioner pipeline schedule	Run grader against submitted work at deadline
<code>update_deadline_schedule</code>	Teacher	Update per-assignment deadline schedule
<code>gather_due_date</code>	Operator (CLI only)	Collect and normalise deadline state

continued on next page

Table A.2 – continued from previous page

Action	Caller role	Effect
lock_main	Operator (CLI only)	Freeze main on a given set of student repositories
enforce_username_policy	Operator (CLI only)	Batch-validate users against the username regex; the live path is the system hook, not this action
run_stresstest	Teacher (optional)	Run stress-test workflow

A.4 Ansible Roles and Host Targets

Role	Host	Purpose
common	all	Base packages, timezone, users; moves admin sshd to 2222 where enabled (greyfox-git)
docker	all	Install and configure Docker Engine
nftables	all	Per-host firewall ruleset
gitlab	greyfox-git	Deploy GitLab CE, bootstrap credentials, application settings
admin_daemon	greyfox-git	Deploy handin-admin-daemon , render env, TLS certs
gitlab_runners	greyfox-ci	Register ci and provisioner runners, tmpfs config
nexus	greyfox-ci	Deploy Nexus, provision Maven/PyPI proxies and Docker Hub proxy endpoint
k8s_base	greyfox-ci	Install k3d and kubect1
k8s_prov	greyfox-ci	k3d cluster for provisioner-side stress-tests
k8s_ci	greyfox-ci	k3d cluster for CI-side stress-tests
web_stack	greyfox-web	Deploy Caddy, backend, frontend, Redis; render deploy script
integration_test	operator control	Run end-to-end tests against the deployed stack

Table A.3: Ansible roles and their target hosts.

A.5 Test Layers

Layer	Test files	Tests	Line coverage
Backend (pytest + respx)	51	634	95% (CI floor 80%)
Frontend (Vitest + Testing Library)	33	228	86% stmts / 88% branch
Provisioner (pytest)	46	843	96%
Ansible integration	48 tasks	live playbook	–

Table A.4: Test-suite size and measured coverage. Backend: `pytest -cov=app` (627 passed, 7 skipped; 95% line coverage). Provisioner: `coverage run -source=scripts -m pytest` (843 passed). Frontend: `npm run test:coverage` (225 passed, 3 skipped); the remaining uncovered statements are concentrated in a few deliberately view-only surfaces (e.g. the hidden branch/reference selector), while the tested logic – hooks, the project picker, provisioning, staff, proctor, claim and status components – sits at or near full coverage. The Ansible layer validates a deployed multi-host stack rather than lines of code, so a coverage percentage does not apply; its 48 task files instead include an anonymous-access sweep asserting every protected endpoint rejects unauthenticated requests.

Layer	Files	Tests	Framework	Measured coverage
Backend unit	51	634	pytest (respx for GitLab)	95% lines ($\geq 80\%$ enforced in CI)
Frontend unit	33	228	Vitest + Testing Library	86% stmts / 88% branch
Provisioner unit	46	843	pytest	96% lines
Integration tasks	48	–	Ansible playbook	Deployed multi-host stack

Table A.5: Test-layer counts, test-case totals, and measured coverage. File counts are current in the checked repository; see [table A.4](#) for the reproduction commands.

A.6 Resource-Use Scenarios

Scenario	CPU	RAM	Internet traffic
Student compile + test job	1–2 cores, bursty for job duration	Per-job cgroup cap; /tmp and /var/tmp scratch in tmpfs; build tree and caches on runner volumes	≈0 warm (image + deps cached); one-time ≈0.5 GiB on a cold image pull
Assignment image publish (teacher)	Moderate (kaniko build)	Moderate, build-scoped	Base-image pull + layer push, up to ≈0.5 GiB; infrequent (once per version)
Provisioning action (create course / mother / child / claim)	Light, short-lived	Light (short Python job)	GitLab API calls on the LAN; negligible external
Invitation-claim burst (≈15–30 students within minutes)	GitLab-bound (membership writes, repo forks)	Backend/Redis rate-limit + idempotency state, small	Small; membership e-mails are suppressed by the SMTP proxy (section 4.7)
Deadline grader batch (all student repos)	N parallel CI jobs, capped by runner concurrency	Per-job caps as above	Internal; no external egress

Table A.6: Estimated dominant resource per run scenario. All figures are engineering estimates from the configured runner caps and image sizes, not production measurements.

Bibliography

- [1] Ansible Community. Ansible documentation. <https://docs.ansible.com/>, 2026. Accessed 2026-07-13.
- [2] Caddy Web Server. Caddy documentation. <https://caddyserver.com/docs/>, 2026. Accessed 2026-07-13.
- [3] Docker, Inc. Docker documentation. <https://docs.docker.com/>, 2026. Accessed 2026-07-13.
- [4] Docker, Inc. Docker security. <https://docs.docker.com/engine/security/>, 2026. Accessed 2026-07-13.
- [5] D. Eastlake and T. Hansen. US secure hash algorithms (SHA and SHA-based HMAC and HKDF). RFC 6234, IETF, 2011.
- [6] Git project. Git reference manual. <https://git-scm.com/docs>, 2026. Accessed 2026-07-13.
- [7] GitLab, Inc. GitLab documentation. <https://docs.gitlab.com/>, 2026. Accessed 2026-07-13.
- [8] GitLab, Inc. CI/CD job token. https://docs.gitlab.com/ee/ci/jobs/ci_job_token.html, 2026. Accessed 2026-07-13.
- [9] GitLab, Inc. GitLab container registry. https://docs.gitlab.com/ee/user/packages/container_registry/, 2026. Accessed 2026-07-13.
- [10] GitLab, Inc. GitLab Runner documentation — docker executor. <https://docs.gitlab.com/runner/executors/docker.html>, 2026. Accessed 2026-07-13.
- [11] GitLab, Inc. System hooks. https://docs.gitlab.com/ee/administration/system_hooks.html, 2026. Accessed 2026-07-13.
- [12] GitLab, Inc. Pipeline trigger tokens. <https://docs.gitlab.com/ee/ci/triggers/>, 2026. Accessed 2026-07-13.
- [13] Google LLC. kaniko — build container images in kubernetes. <https://github.com/GoogleContainerTools/kaniko>, 2026. Accessed 2026-07-13.

- [14] D. Hardt. The OAuth 2.0 authorization framework. RFC 6749, IETF, October 2012. URL <https://www.rfc-editor.org/info/rfc6749>.
- [15] H. Krawczyk, M. Bellare, and R. Canetti. HMAC: Keyed-Hashing for message authentication. RFC 2104, IETF, February 1997. URL <https://www.rfc-editor.org/info/rfc2104>.
- [16] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett. OAuth 2.0 security best current practice. Internet-Draft `draft-ietf-oauth-security-topics`, IETF, 2024. Work in progress; version referenced captured on 2026-07-13.
- [17] Meta Open Source. React documentation. <https://react.dev/>, 2026. Accessed 2026-07-13.
- [18] Microsoft. Monaco Editor. <https://microsoft.github.io/monaco-editor/>, 2026. Accessed 2026-07-13; the code editor that powers VS Code.
- [19] OWASP Foundation. OWASP Top 10:2021. <https://owasp.org/Top10/>, 2021. Accessed 2026-07-13.
- [20] OWASP Foundation. Cross-site request forgery prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html, 2024. Accessed 2026-07-13.
- [21] OWASP Foundation. Session management cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html, 2024. Accessed 2026-07-13.
- [22] S. Ramírez. FastAPI documentation. <https://fastapi.tiangolo.com/>, 2026. Accessed 2026-07-13.
- [23] Redis Ltd. Redis documentation. <https://redis.io/docs/>, 2026. Accessed 2026-07-13.
- [24] Sonatype, Inc. Nexus Repository documentation. <https://help.sonatype.com/repomanager3>, 2026. Accessed 2026-07-13.